

Strict I/O False Positives in TritonBench L40S

This report is generated from the clean full strict I/O rerun. It focuses on cases where the repository-style stdout oracle accepts the generated file, but the stricter checker finds missing or mismatched structured outputs.

field	value
run directory	/private/tmp/tritonbench-l40s-inspect/experiments/tritonbench_l40s_20260518_184231
weak-oracle cases	22
matched candidates	234
stdout false positives	156
strict failures	160
import errors	74

0.1. How To Read The Evidence

For each task, TritonBench has a gold source file whose test tail builds `result_gold`, `results`, or `test_results`. The weak I/O oracle compares only captured stdout. Many gold tests do not print the result dictionary, and many materialized generated files also print nothing. Therefore `stdout == stdout` can be true even when the generated implementation returns nothing, exposes no result global, or computes different tensors.

The strict rerun imports the generated file and the gold file with a fixed seed, normalizes result globals, and compares them. The common mismatch `$: type differs: NoneType != dict` means the candidate exposed no structured result while the gold test produced a dictionary.

0.2. G / fused_recurrent_retention

field	value
speedup	1.6245
stdout false-positive candidates	11
representative candidate	Bench_G_general_purpose/output_claude-3-5-sonnet-20240620_simp/candidate_0095.py
strict mismatch	<code>\$: type differs: NoneType != dict</code>
candidate stdout	<code>''</code>
reference stdout	<code>''</code>

0.2.1. GT Implementation Snippet

```
import torch
import triton
import triton.language as tl
from typing import Tuple

@triton.jit
def fused_recurrent_retention_fwd_kernel(
    q, k, v, o, initial_state, final_state,
    s_qk_h, s_qk_t, s_qk_d,
    s_vo_h, s_vo_t, s_vo_d,
    B, H, T, scale,
    BK: tl.constexpr, BV: tl.constexpr, DK: tl.constexpr, DV: tl.constexpr,
    USE_INITIAL_STATE: tl.constexpr, STORE_FINAL_STATE: tl.constexpr
):
    # Kernel logic
    i_v, i_k, i_bh = tl.program_id(0), tl.program_id(1), tl.program_id(2)
    i_h = i_bh % H
    b_b = (1 - tl.math.exp2(-5 - i_h * 1.0))

    p_q = q + i_bh * s_qk_h + i_k * BK + tl.arange(0, BK)
    p_k = k + i_bh * s_qk_h + i_k * BK + tl.arange(0, BK)
    p_v = v + i_bh * s_vo_h + i_v * BV + tl.arange(0, BV)
    p_o = o + (i_bh + i_k * B * H) * s_vo_h + i_v * BV + tl.arange(0, BV)

    mask_bk = (i_k * BK + tl.arange(0, BK)) < DK
    mask_bv = (i_v * BV + tl.arange(0, BV)) < DV
    mask_kv = mask_bk[None, :] & mask_bv[:, None]

    h = tl.zeros([BV, BK], dtype=tl.float32)

    if USE_INITIAL_STATE:
        p_init_s = initial_state + i_bh * DK * DV + \
```

```

        (i_k * BK + tl.arange(0, BK)[None, :]) * \
        DV + (i_v * BV + tl.arange(0, BV)[: , None])
    h += tl.load(p_init_s, mask=mask_kv, other=0).to(tl.float32)

for _ in range(0, T):
    _k = tl.load(p_k, mask=mask_bk, other=0).to(tl.float32)
    _v = tl.load(p_v, mask=mask_bv, other=0).to(tl.float32)
    _q = tl.load(p_q, mask=mask_bk, other=0).to(tl.float32) * scale

    h = b_b * h + _k[None, :] * _v[:, None]
    _o = h * _q[None, :]
    _o = tl.sum(_o, axis=1)
    tl.store(p_o, _o.to(p_o.dtype.element_ty), mask=mask_bv)

    p_q += DK
    p_k += DK
    p_o += DV
    p_v += DV

if STORE_FINAL_STATE:
    p_final_s = final_state + i_bh * DK * DV + \
        (i_k * BK + tl.arange(0, BK)[None, :]) * \
        DV + (i_v * BV + tl.arange(0, BV)[: , None])
    tl.store(p_final_s, h.to(p_final_s.dtype.element_ty), mask=mask_kv)

@triton.jit
def fused_recurrent_retention_bwd_kernel(
    q, k, v, do, dq, dk, dv, initial_state,
    s_qk_h, s_qk_t, s_qk_d,
    s_vo_h, s_vo_t, s_vo_d,
    B, H, T, scale,
    BK: tl.constexpr, BV: tl.constexpr, DK: tl.constexpr, DV: tl.constexpr,
    USE_INITIAL_STATE: tl.constexpr
):
    # Kernel logic
    i_v, i_k, i_bh = tl.program_id(0), tl.program_id(1), tl.program_id(2)
    i_h = i_bh % H
    b_b = 1 - tl.math.exp2(-5 - i_h * 1.0)
    ... <truncated; 145 more lines>

```

0.2.2. Generated Solution Snippet

```

import torch
import triton
import triton.language as tl

@triton.jit
def fused_recurrent_retention_forward_kernel(
    # Pointers to tensors
    q_ptr, k_ptr, v_ptr, o_ptr, h_ptr,
    initial_state_ptr, final_state_ptr,
    # Dimensions
    B, H, T, D,
    BK: tl.constexpr, BV: tl.constexpr,
    USE_INITIAL_STATE: tl.constexpr,
    STORE_FINAL_STATE: tl.constexpr,
    stride_qb, stride_qh, stride_qt, stride_qd,
    stride_kb, stride_kh, stride_kt, stride_kd,
    stride_vb, stride_vh, stride_vt, stride_vd,
    stride_ob, stride_oh, stride_ot, stride_od,
    BLOCK_SIZE: tl.constexpr):

    # Compute indices
    pid = tl.program_id(0)
    num_pid_b = tl.cdiv(B, BLOCK_SIZE)
    num_pid_h = H
    num_pid_t = 1

    pid_b = pid // (num_pid_h * num_pid_t)
    pid_h = (pid % (num_pid_h * num_pid_t)) // num_pid_t

    # Initialize h state
    h = tl.zeros([BK, BV], dtype=tl.float32)

```

```

if USE_INITIAL_STATE:
    offs_init = pid_b * stride_qb + pid_h * stride_qh
    h = tl.load(initial_state_ptr + offs_init)

# Block pointers
offs_q = pid_b * stride_qb + pid_h * stride_qh
offs_k = pid_b * stride_kb + pid_h * stride_kh
offs_v = pid_b * stride_vb + pid_h * stride_vh
offs_o = pid_b * stride_ob + pid_h * stride_oh

# Loop over sequence length
for t in range(T):
    # Load q, k, v blocks
    q = tl.load(q_ptr + offs_q + t * stride_qt)
    k = tl.load(k_ptr + offs_k + t * stride_kt)
    v = tl.load(v_ptr + offs_v + t * stride_vt)

    # Update h state
    h = h + tl.dot(k, v)

    # Compute output
    o = tl.dot(q, h)

    # Store output
    tl.store(o_ptr + offs_o + t * stride_ot, o)

# Store final state if needed
if STORE_FINAL_STATE:
    offs_final = pid_b * stride_qb + pid_h * stride_qh
    tl.store(final_state_ptr + offs_final, h)

@triton.jit
def fused_recurrent_retention_backward_kernel(
    # Pointers to tensors
    dq_ptr, dk_ptr, dv_ptr, do_ptr,
    q_ptr, k_ptr, v_ptr, h_ptr,
    # Dimensions and strides similar to forward kernel
    B, H, T, D,
    BK: tl.constexpr, BV: tl.constexpr,
    stride_qb, stride_qh, stride_qt, stride_qd,
    stride_kb, stride_kh, stride_kt, stride_kd,
    stride_vb, stride_vh, stride_vt, stride_vd,
    BLOCK_SIZE: tl.constexpr):

    pid = tl.program_id(0)
    num_pid_b = tl.cdiv(B, BLOCK_SIZE)
    num_pid_h = H

    pid_b = pid // num_pid_h
    pid_h = pid % num_pid_h

    # Initialize gradients
    dh = tl.zeros([BK, BV], dtype=tl.float32)

    # Block pointers
    offs_q = pid_b * stride_qb + pid_h * stride_qh
    offs_k = pid_b * stride_kb + pid_h * stride_kh
    offs_v = pid_b * stride_vb + pid_h * stride_vh

... <truncated; 86 more lines>

```

0.2.3. I/O Test Inputs That Are Treated As Passing

The weak oracle sees this as passing because the generated script and the gold script have identical stdout. The recovered gold test tail below shows the inputs and result dictionary that should have been compared structurally.

```

import torch

# Extended test function with backward propagation
def test_fused_recurrent_retention_with_backward():
    test_results = {}

    # Test parameters

```

```

batch_size = 2
n_heads = 4
seq_len = 8
d_head_qk = 16
d_head_v = 16

# Create random input tensors
q = torch.randn(batch_size, n_heads, seq_len, d_head_qk, dtype=torch.float32, requires_grad=True,
device='cuda')
k = torch.randn(batch_size, n_heads, seq_len, d_head_qk, dtype=torch.float32, requires_grad=True,
device='cuda')
v = torch.randn(batch_size, n_heads, seq_len, d_head_v, dtype=torch.float32, requires_grad=True,
device='cuda')

# Test 1: Without initial state and without final state
initial_state = None
output_final_state = False
o, final_state = fused_recurrent_retention(q, k, v, initial_state=initial_state,
output_final_state=output_final_state)
loss = o.sum() # Define a simple loss function
loss.backward() # Perform backward pass
test_results['test_case_1'] = {
    "output_shape": o.shape,
    "final_state": final_state,
    "loss": loss.item(),
    "gradients_q": q.grad.norm().item(),
    "gradients_k": k.grad.norm().item(),
    "gradients_v": v.grad.norm().item()
}

# Reset gradients for the next test
q.grad.zero_()
k.grad.zero_()
v.grad.zero_()

# Test 2: With initial state and without final state
initial_state = torch.randn(batch_size, n_heads, d_head_qk, d_head_v, dtype=torch.float32, device='cuda',
requires_grad=True)
o, final_state = fused_recurrent_retention(q, k, v, initial_state=initial_state, output_final_state=False)
loss = o.sum()
loss.backward()
test_results['test_case_2'] = {
    "output_shape": o.shape,
    "final_state": final_state,
    "loss": loss.item(),
    "gradients_q": q.grad.norm().item(),
    "gradients_k": k.grad.norm().item(),
    "gradients_v": v.grad.norm().item(),
}

# Reset gradients for the next test
q.grad.zero_()
k.grad.zero_()
v.grad.zero_()

# Test 3: With initial state and with final state
o, final_state = fused_recurrent_retention(q, k, v, initial_state=initial_state, output_final_state=True)
loss = o.sum() + final_state.sum()
loss.backward()
test_results['test_case_3'] = {
    "output_shape": o.shape,
    "final_state_shape": final_state.shape,
    "loss": loss.item(),
    "gradients_q": q.grad.norm().item(),
    "gradients_k": k.grad.norm().item(),
    "gradients_v": v.grad.norm().item()
}

# Test 4: Without initial state and with final state
initial_state = None
output_final_state = True
o, final_state = fused_recurrent_retention(q, k, v, initial_state=initial_state,

```

```

output_final_state=output_final_state)
    loss = o.sum() + final_state.sum()
    loss.backward()
    test_results['test_case_4'] = {
        "output_shape": o.shape,
        "final_state_shape": final_state.shape,
        "loss": loss.item(),
        "gradients_q": q.grad.norm().item(),
        "gradients_k": k.grad.norm().item(),
        "gradients_v": v.grad.norm().item()
    }

    return test_results

# Run the test function with backward propagation
result_gold = test_fused_recurrent_retention_with_backward()

```

0.2.4. Why The I/O Test Passes

- candidate stdout: ''
- reference stdout: ''
- the stdout strings are equal, so the weak oracle accepts the generated file;
- strict comparison then reports: \$: type differs: NoneType != dict.

0.2.5. Why It Is Not Correct

Several generated implementations use the wrong recurrence decay or the wrong public API. The gold kernel uses $1 - 2^{-(5 - \text{head})}$, while representative generated code uses $1 / 2^{(\text{head} + 1)}$ or renames/misses output_final_state.

0.2.6. Obvious Counterexample / Follow-up Test

Call fused_recurrent_retention(q, k, v, initial_state=None, output_final_state=True) and compare (output, final_state). For head 0, a decay of 0.5 instead of 0.96875 causes a large output/final-state mismatch.

0.2.7. All Stdout-False-Positive Candidates For This Task

- Bench_G_coder_specofic/qwen_092/candidate_0095.py: \$: type differs: NoneType != dict
- Bench_G_general_purpose/Qwen2.5-72B-Instruct_comp/candidate_0095.py: \$: type differs: NoneType != dict
- Bench_G_general_purpose/output_DeepSeek-R1_comp/candidate_0095.py: \$: type differs: NoneType != dict
- Bench_G_general_purpose/output_DeepSeek-R1_comp_/candidate_0095.py: \$: type differs: NoneType != dict
- Bench_G_general_purpose/output_DeepSeek-R1_comp_rag/candidate_0085.py: \$: type differs: NoneType != dict
- Bench_G_general_purpose/output_DeepSeek-R1_comp_rag_/candidate_0099.py: \$: type differs: NoneType != dict
- Bench_G_general_purpose/output_claude-3-5-sonnet-20240620_comp/candidate_0095.py: \$: type differs: NoneType != dict
- Bench_G_general_purpose/output_claude-3-5-sonnet-20240620_comp_rag/candidate_0099.py: \$: type differs: NoneType != dict
- Bench_G_general_purpose/output_claude-3-5-sonnet-20240620_simp/candidate_0095.py: \$: type differs: NoneType != dict
- Bench_G_general_purpose/output_gpt-4o_simp/candidate_0095.py: \$: type differs: NoneType != dict
- Bench_G_general_purpose/output_o1-2024-12-17_comp_rag_/candidate_0099.py: \$: type differs: NoneType != dict

0.3. G / l2_norm_bwd

field	value
speedup	1.4409
stdout false-positive candidates	2
representative candidate	Bench_G_general_purpose/output_claude-3-5-sonnet-20240620_simp/candidate_0175.py
strict mismatch	\$: type differs: NoneType != dict
candidate stdout	''
reference stdout	''

0.3.1. GT Implementation Snippet

```
import torch
import triton
import triton.language as tl

@triton.jit
def _l2_norm_bwd_kernel(
    X, # pointer to the input
    DY, # pointer to the output gradient
    DX, # pointer to the input gradient
    stride_x_row, # how much to increase the pointer when moving by 1 row
    N, # number of columns in X
    eps, # epsilon to avoid division by zero
    BLOCK_N: tl.constexpr,
):
    row = tl.program_id(0)
    X += row * stride_x_row
    DX += row * stride_x_row
    DY += row * stride_x_row
    cols = tl.arange(0, BLOCK_N)
    x = tl.load(X + cols, mask=cols < N, other=0.0).to(tl.float32)
    x = tl.where(cols < N, x, 0.0)
    var = tl.sum(x * x)
    rstd = 1 / tl.sqrt(var + eps)
    mask = cols < N
    dy = tl.load(DY + cols, mask=cols < N, other=0.0).to(tl.float32)
    dy = tl.where(cols < N, dy, 0.0)
    dx = dy * rstd - tl.sum(dy * x) * (1 / (var+eps)) * rstd * x
    tl.store(DX + cols, dx, mask=mask)

def _l2_norm_bwd(
    x, dy, eps=1e-5,
):
    x_shape_og = x.shape
    x = x.reshape(-1, dy.shape[-1])
    dy = dy.reshape(-1, dy.shape[-1])
    if dy.stride(-1) != 1:
        dy = dy.contiguous()
    dx = torch.empty_like(x)
    N = x.shape[-1]
    M = x.shape[0]
    MAX_FUSED_SIZE = 65536 // x.element_size()
    BLOCK_N = min(MAX_FUSED_SIZE, triton.next_power_of_2(N))
    if N > BLOCK_N:
        raise RuntimeError(
            "This layer norm doesn't support feature dim >= 64KB."
        )
    with torch.cuda.device(x.device.index):
        _l2_norm_bwd_kernel[(M,)](
            x,
            dy,
            dx,
            x.stride(0),
            N,
            eps,
            BLOCK_N,
        )
    return dx.reshape(x_shape_og)
```

0.3.2. Generated Solution Snippet

```
import triton
import triton.language as tl
import torch

@triton.jit
def _l2_norm_bwd_kernel(
    X_ptr, DY_ptr, DX_ptr,
    stride_xm, stride_xn,
    stride_dym, stride_dyn,
    stride_dxm, stride_dxn,
    M, N,
    eps,
    BLOCK_SIZE: tl.constexpr,
```

```

):
    # Program ID
    pid = tl.program_id(0)

    # Compute row index
    row = pid

    # Pointers for the current row
    x_row_ptr = X_ptr + row * stride_xm
    dy_row_ptr = DY_ptr + row * stride_dym
    dx_row_ptr = DX_ptr + row * stride_dxm

    # Load row data
    cols = tl.arange(0, BLOCK_SIZE)
    mask = cols < N

    x = tl.load(x_row_ptr + cols * stride_xn, mask=mask, other=0.0)
    dy = tl.load(dy_row_ptr + cols * stride_dyn, mask=mask, other=0.0)

    # Compute L2 norm
    x_squared = x * x
    row_sum = tl.sum(x_squared, axis=0)
    norm = tl.sqrt(row_sum + eps)

    # Compute gradients
    norm_inv = 1.0 / norm
    norm_inv_cube = norm_inv * norm_inv * norm_inv

    dot_product = tl.sum(x * dy, axis=0)

    # Backward formula: dx = (dy/norm - x * dot_product/norm^3)
    dx = norm_inv * dy - (x * dot_product * norm_inv_cube)

    # Store results
    tl.store(dx_row_ptr + cols * stride_dxn, dx, mask=mask)

def _l2_norm_bwd(x: torch.Tensor, dy: torch.Tensor, eps: float = 1e-12) -> torch.Tensor:
    # Get input dimensions
    M, N = x.shape

    # Allocate output
    dx = torch.empty_like(x)

    # Define block size (can be tuned)
    BLOCK_SIZE = triton.next_power_of_2(N)

    # Launch kernel
    grid = (M,)

    _l2_norm_bwd_kernel[grid](
        x, dy, dx,
        x.stride(0), x.stride(1),
        dy.stride(0), dy.stride(1),
        dx.stride(0), dx.stride(1),
        M, N,
        eps,
        BLOCK_SIZE=BLOCK_SIZE,
    )

    return dx

# Example usage
def l2_norm_bwd(x: torch.Tensor, dy: torch.Tensor, eps: float = 1e-12) -> torch.Tensor:
    """
    Compute backward pass of L2 normalization.

    Args:
        x: Input tensor of shape (M, N)
        dy: Gradient tensor of shape (M, N)
        eps: Small constant for numerical stability

    Returns:

```



```

        dx: Gradient tensor of shape (M, N)
    """
    assert x.dim() == 2, "Input must be 2-dimensional"
    assert x.shape == dy.shape, "Input and gradient must have the same shape"
    assert x.is_cuda and dy.is_cuda, "Inputs must be CUDA tensors"
... <truncated; 2 more lines>

```

0.3.3. I/O Test Inputs That Are Treated As Passing

The weak oracle sees this as passing because the generated script and the gold script have identical stdout. The recovered gold test tail below shows the inputs and result dictionary that should have been compared structurally.

```

import torch

# Test the backward L2 normalization
def test_l2_norm_bwd():
    results = {}

    # Test case 1: Default case
    x = torch.randn(4, 8, device='cuda', dtype=torch.float32)
    dy = torch.randn(4, 8, device='cuda', dtype=torch.float32)
    dx = _l2_norm_bwd(x, dy)
    results['test_case_1'] = dx

    # Test case 2: Different shape
    x = torch.randn(2, 16, device='cuda', dtype=torch.float32)
    dy = torch.randn(2, 16, device='cuda', dtype=torch.float32)
    dx = _l2_norm_bwd(x, dy)
    results['test_case_2'] = dx

    # Test case 3: Larger tensor
    x = torch.randn(8, 8, device='cuda', dtype=torch.float32)
    dy = torch.randn(8, 8, device='cuda', dtype=torch.float32)
    dx = _l2_norm_bwd(x, dy)
    results['test_case_3'] = dx

    # Test case 4: Edge case with small tensor
    x = torch.randn(1, 8, device='cuda', dtype=torch.float32)
    dy = torch.randn(1, 8, device='cuda', dtype=torch.float32)
    dx = _l2_norm_bwd(x, dy)
    results['test_case_4'] = dx

    return results

# Run the tests
result_gold = test_l2_norm_bwd()

```

0.3.4. Why The I/O Test Passes

- candidate stdout: ''
- reference stdout: ''
- the stdout strings are equal, so the weak oracle accepts the generated file;
- strict comparison then reports: \$: type differs: NoneType != dict.

0.3.5. Why It Is Not Correct

Representative false-positive candidates expose the wrong function name/signature. The gold API is `_l2_norm_bwd(x, dy, eps=1e-5)`, while one candidate defines `l2_norm_bwd(X, DY, variance, rstd, BLOCK_SIZE=256)`. Other generated variants reverse `(x, dy)` or use a different default epsilon.

0.3.6. Obvious Counterexample / Follow-up Test

Call `_l2_norm_bwd(torch.randn(1, 8, device='cuda'), torch.randn(1, 8, device='cuda'))`; candidates without `_l2_norm_bwd` fail immediately. For epsilon bugs, use a near-zero row such as `x = full((2, 8), 1e-8)`.

0.3.7. All Stdout-False-Positive Candidates For This Task

- Bench_G_coder_specofic/qwen_092/candidate_0046.py: \$: type differs: NoneType != dict
- Bench_G_general_purpose/output_claude-3-5-sonnet-20240620_simp/candidate_0175.py: \$: type differs: NoneType != dict

0.4. G / triton_mul2

field	value
-------	-------

speedup	1.3112
stdout false-positive candidates	20
representative candidate	Bench_G_coder_specofic/qwen_tune_300/candidate_0148.py
strict mismatch	\$. type differs: NoneType != dict
candidate stdout	''
reference stdout	''

0.4.1. GT Implementation Snippet

```
import triton
from triton import language as tl
import torch
```

```
@triton.jit
def mul2_kernel(
    in_ptr0,
    out_ptr,
    n_elements,
    BLOCK_SIZE: "tl.constexpr",
):
    pid = tl.program_id(axis=0)
    block_start = pid * BLOCK_SIZE
    offsets = block_start + tl.arange(0, BLOCK_SIZE)
    mask = offsets < n_elements
    x = tl.load(in_ptr0 + offsets, mask=mask)
    output = 2 * x
    tl.store(out_ptr + offsets, output, mask=mask)
```

```
@triton.jit
def mul2_inplace_kernel(
    ptr,
    n_elements,
    BLOCK_SIZE: "tl.constexpr",
):
    pid = tl.program_id(axis=0)
    block_start = pid * BLOCK_SIZE
    offsets = block_start + tl.arange(0, BLOCK_SIZE)
    mask = offsets < n_elements
    x = tl.load(ptr + offsets, mask=mask)
    output = 2 * x
    tl.store(ptr + offsets, output, mask=mask)
```

```
def triton_mul2(x, BLOCK_SIZE=16):
    output = torch.zeros_like(x)
    n_elements = x.numel()
    grid = (triton.cdiv(n_elements, BLOCK_SIZE),)
    mul2_kernel[grid](x, output, n_elements, BLOCK_SIZE)
    return output
```

```
def triton_mul2_inplace(x, BLOCK_SIZE=16):
    n_elements = x.numel()
    grid = (triton.cdiv(n_elements, BLOCK_SIZE),)
    mul2_inplace_kernel[grid](x, n_elements, BLOCK_SIZE)
    return x
```

0.4.2. Generated Solution Snippet

```
import torch
import triton
import triton.language as tl
```

```
@triton.jit
def mul2_kernel(
    input,
    output,
    n_elements,
    BLOCK_SIZE: tl.constexpr,
):
```

```

pid = tl.program_id(axis=0)
block_start = pid * BLOCK_SIZE
offsets = block_start + tl.arange(0, BLOCK_SIZE)
mask = offsets < n_elements
x = tl.load(input + offsets, mask=mask)
x = x * 2
tl.store(output + offsets, x, mask=mask)

@triton.jit
def mul2_inplace_kernel(
    input,
    n_elements,
    BLOCK_SIZE: tl.constexpr,
):
    pid = tl.program_id(axis=0)
    block_start = pid * BLOCK_SIZE
    offsets = block_start + tl.arange(0, BLOCK_SIZE)
    mask = offsets < n_elements
    x = tl.load(input + offsets, mask=mask)
    x = x * 2
    tl.store(input + offsets, x, mask=mask)

def triton_mul2(input):
    output = torch.empty_like(input)
    n_elements = output.numel()
    grid = lambda meta: (triton.cdiv(n_elements, meta['BLOCK_SIZE']),)
    mul2_kernel[grid](input, output, n_elements, BLOCK_SIZE=1024)
    return output

def triton_mul2_inplace(input):
    n_elements = input.numel()
    grid = lambda meta: (triton.cdiv(n_elements, meta['BLOCK_SIZE']),)
    mul2_inplace_kernel[grid](input, n_elements, BLOCK_SIZE=1024)
    return input

```

0.4.3. I/O Test Inputs That Are Treated As Passing

The weak oracle sees this as passing because the generated script and the gold script have identical stdout. The recovered gold test tail below shows the inputs and result dictionary that should have been compared structurally.

```

def test_mul():
    N = 1024 * 1024
    x = torch.randn(N, device='cuda')

    # 分支1: triton_mul2 with BLOCK_SIZE=1024
    triton_mul2_result = triton_mul2(x, BLOCK_SIZE=1024)

    # 分支2: triton_mul2_inplace with BLOCK_SIZE=1024
    triton_mul2_inplace_result = triton_mul2_inplace(x.clone(), BLOCK_SIZE=1024)

    # 分支3: triton_mul2 with a different BLOCK_SIZE
    triton_mul2_result_case2 = triton_mul2(x, BLOCK_SIZE=512)

    # 分支4: triton_mul2_inplace with a different BLOCK_SIZE
    triton_mul2_inplace_result_case2 = triton_mul2_inplace(x.clone(), BLOCK_SIZE=512)

    # 返回测试结果
    result_dict = {
        "test_case_1": triton_mul2_result,
        "test_case_2": triton_mul2_inplace_result,
        "test_case_3": triton_mul2_result_case2,
        "test_case_4": triton_mul2_inplace_result_case2,
    }

    return result_dict

# 执行测试函数
result_gold = test_mul()

```

0.4.4. Why The I/O Test Passes

- candidate stdout: ''
- reference stdout: ''

- the stdout strings are equal, so the weak oracle accepts the generated file;
- strict comparison then reports: \$: type differs: NoneType != dict.

0.4.5. Why It Is Not Correct

Representative false-positive candidates expose the wrong public API. The gold function is `triton_mul2(x, BLOCK_SIZE=16)` and returns a tensor. A generated candidate defines `triton_mul2(x, y)`, requires caller-provided output, and returns nothing.

0.4.6. Obvious Counterexample / Follow-up Test

Call `triton_mul2(torch.tensor([1., -3.], device='cuda'), BLOCK_SIZE=2)` and expect `[2., -6.]`. The representative candidate raises a signature error or returns `None`.

0.4.7. All Stdout-False-Positive Candidates For This Task

- `Bench_G_coder_specofic/deepseek_094/candidate_0148.py`: \$: type differs: NoneType != dict
- `Bench_G_coder_specofic/deepseek_rag_145/candidate_0148.py`: \$: type differs: NoneType != dict
- `Bench_G_coder_specofic/deepseek_rag_303/candidate_0148.py`: \$: type differs: NoneType != dict
- `Bench_G_coder_specofic/deepseek_tune_255/candidate_0148.py`: \$: type differs: NoneType != dict
- `Bench_G_coder_specofic/qwen_rag_145/candidate_0148.py`: \$: type differs: NoneType != dict
- `Bench_G_coder_specofic/qwen_tune_300/candidate_0148.py`: \$: type differs: NoneType != dict
- `Bench_G_coder_specofic/qwen_tune_rag_250/candidate_0148.py`: \$: type differs: NoneType != dict
- `Bench_G_general_purpose/Qwen2.5-72B-Instruct_comp/candidate_0148.py`: \$: type differs: NoneType != dict
- `Bench_G_general_purpose/output_DeepSeek-R1_comp/candidate_0148.py`: \$: type differs: NoneType != dict
- `Bench_G_general_purpose/output_DeepSeek-R1_comp_/candidate_0148.py`: \$: type differs: NoneType != dict
- `Bench_G_general_purpose/output_DeepSeek-R1_comp_rag/candidate_0151.py`: \$: type differs: NoneType != dict
- `Bench_G_general_purpose/output_DeepSeek-R1_comp_rag_/candidate_0140.py`: \$: type differs: NoneType != dict
- `Bench_G_general_purpose/output_claude-3-5-sonnet-20240620_comp/candidate_0148.py`: \$: type differs: NoneType != dict
- `Bench_G_general_purpose/output_claude-3-5-sonnet-20240620_comp_rag/candidate_0140.py`: \$: type differs: NoneType != dict
- `Bench_G_general_purpose/output_claude-3-5-sonnet-20240620_simp/candidate_0148.py`: \$: type differs: NoneType != dict
- `Bench_G_general_purpose/output_claude-3-5-sonnet-20240620_simp_rag/candidate_0144.py`: \$: type differs: NoneType != dict
- `Bench_G_general_purpose/output_gpt-4o_comp/candidate_0148.py`: \$: type differs: NoneType != dict
- `Bench_G_general_purpose/output_gpt-4o_comp_rag/candidate_0156.py`: \$: type differs: NoneType != dict
- `Bench_G_general_purpose/output_gpt-4o_simp_rag/candidate_0143.py`: \$: type differs: NoneType != dict
- `Bench_G_general_purpose/output_o1-2024-12-17_comp/candidate_0148.py`: \$: type differs: NoneType != dict

0.5. T / broadcast_tensors

field	value
speedup	2.7575
stdout false-positive candidates	12
representative candidate	<code>Bench_T_general_purpose/output_o1-2024-12-17_rag/broadcast_tensors.py</code>
strict mismatch	<code>\$: type differs: NoneType != dict</code>
candidate stdout	<code>''</code>
reference stdout	<code>''</code>

0.5.1. GT Implementation Snippet

```
import torch

def broadcast_tensors(x, y):
    (a, b) = torch.broadcast_tensors(x, y)
    return (a, b)
```

0.5.2. Generated Solution Snippet

```
import triton
import triton.language as tl
import torch

@triton.jit
def _broadcast_tensors_kernel():
    # This is a no-op kernel included for demonstration.
    # Broadcasting in PyTorch is primarily a shape/view operation,
    # so no actual memory operation is necessary here.
    tl.debug_barrier()

def broadcast_tensors(*tensors):
    # Check that all input tensors share the same dtype (optional check).
    dtypes = [t.dtype for t in tensors]
    if not all(dtype == dtypes[0] for dtype in dtypes):
        raise ValueError("All input tensors must have the same dtype.")

    # Compute the broadcast shape.
    broadcast_shape = torch.broadcast_shapes(*(t.shape for t in tensors))

    # Return new views of the original tensors with broadcasted shapes.
    # Note that these views may point to the same memory locations
    # for dimensions of size 1, so in-place writes can be unsafe.
    broadcasted = [t.expand(broadcast_shape) for t in tensors]

    # Optionally call a dummy Triton kernel (no-op).
    _broadcast_tensors_kernel.run()

    return broadcasted
```

0.5.3. I/O Test Inputs That Are Treated As Passing

The weak oracle sees this as passing because the generated script and the gold script have identical stdout. The recovered gold test tail below shows the inputs and result dictionary that should have been compared structurally.

```
import torch

def test_broadcast_tensors():
    results = {}

    # Test case 1: Broadcasting a scalar and a 1D tensor
    x1 = torch.tensor(3.0, device='cuda')
    y1 = torch.tensor([1.0, 2.0, 3.0], device='cuda')
    results["test_case_1"] = broadcast_tensors(x1, y1)

    # Test case 2: Broadcasting two 1D tensors of different sizes
    x2 = torch.tensor([1.0, 2.0, 3.0], device='cuda')
    y2 = torch.tensor([1.0], device='cuda')
    results["test_case_2"] = broadcast_tensors(x2, y2)

    # Test case 3: Broadcasting a 2D tensor and a 1D tensor
    x3 = torch.tensor([[1.0, 2.0, 3.0]], device='cuda')
    y3 = torch.tensor([1.0, 2.0, 3.0], device='cuda')
    results["test_case_3"] = broadcast_tensors(x3, y3)

    # Test case 4: Broadcasting two 2D tensors of different shapes
    x4 = torch.tensor([[1.0], [2.0], [3.0]], device='cuda')
    y4 = torch.tensor([[1.0, 2.0, 3.0]], device='cuda')
    results["test_case_4"] = broadcast_tensors(x4, y4)

    return results

test_results = test_broadcast_tensors()
```

0.5.4. Why The I/O Test Passes

- candidate stdout: ''
- reference stdout: ''
- the stdout strings are equal, so the weak oracle accepts the generated file;
- strict comparison then reports: \$: type differs: NoneType != dict.

0.5.5. Why It Is Not Correct

The gold is `torch.broadcast_tensors(x, y)`. A representative false-positive candidate calls `tl.broadcast_tensors` without importing `tl`; Triton language also has no Python-level replacement with PyTorch's view/stride semantics.

0.5.6. Obvious Counterexample / Follow-up Test

Use `x = torch.tensor(3.0, device='cuda')`, `y = torch.tensor([1., 2., 3.], device='cuda')`; the gold returns two broadcasted tensors, while the representative candidate raises `NameError` or lacks the API.

0.5.7. All Stdout-False-Positive Candidates For This Task

- `Bench_T_coder_specific/deepseek/broadcast_tensors.py`: \$: type differs: NoneType != dict
- `Bench_T_coder_specific/deepseek_full/broadcast_tensors.py`: \$: type differs: NoneType != dict
- `Bench_T_coder_specific/deepseek_rag/broadcast_tensors.py`: \$: type differs: NoneType != dict
- `Bench_T_coder_specific/qwen_full/broadcast_tensors.py`: \$: type differs: NoneType != dict
- `Bench_T_coder_specific/qwen_full_rag/broadcast_tensors.py`: \$: type differs: NoneType != dict
- `Bench_T_coder_specific/qwen_rag/broadcast_tensors.py`: \$: type differs: NoneType != dict
- `Bench_T_general_purpose/output_DeepSeek-R1/broadcast_tensors.py`: \$: type differs: NoneType != dict
- `Bench_T_general_purpose/output_DeepSeek-R1_rag/broadcast_tensors.py`: \$: type differs: NoneType != dict
- `Bench_T_general_purpose/output_claude-3-5-sonnet-20240620/broadcast_tensors.py`: \$: type differs: NoneType != dict
- `Bench_T_general_purpose/output_claude-3-5-sonnet-20240620_rag/broadcast_tensors.py`: \$: type differs: NoneType != dict
- `Bench_T_general_purpose/output_o1-2024-12-17/broadcast_tensors.py`: \$: type differs: NoneType != dict
- `Bench_T_general_purpose/output_o1-2024-12-17_rag/broadcast_tensors.py`: \$: type differs: NoneType != dict

0.6. T / det

field	value
speedup	1.5777
stdout false-positive candidates	10
representative candidate	<code>Bench_T_coder_specific/qwen_full/linalg.py</code>
strict mismatch	<code>\$: type differs: NoneType != dict</code>
candidate stdout	<code>''</code>
reference stdout	<code>''</code>

0.6.1. GT Implementation Snippet

```
import torch

def det(A):
    return torch.linalg.det(A)
```

0.6.2. Generated Solution Snippet

```
import torch
import triton
import triton.language as tl

@triton.jit
def _det(A):
    # Implementation details omitted for brevity
    pass

def det(A, *, out=None):
    # Wrapper function for computing the determinant of a square matrix using Triton
    if A.is_floating_point():
        if A.dtype in (torch.float32, torch.float64):
            # For float32 and float64, use default grid and stream
            return _det(A, grid=A.shape, stream=None)
        elif A.dtype in (torch.complex64, torch.complex128):
            # For complex64 and complex128, compute the determinant of the matrix's absolute value
            return torch.prod(torch.abs(A), dim=(-2, -1)).to(A)
    else:
```

```
# Raise an error if the input tensor is not of floating point type
raise RuntimeError("torch.det only supports floating-point inputs")
```

0.6.3. I/O Test Inputs That Are Treated As Passing

The weak oracle sees this as passing because the generated script and the gold script have identical stdout. The recovered gold test tail below shows the inputs and result dictionary that should have been compared structurally.

```
import torch

# def det(A):
#     return torch.linalg.det(A)

def test_det():
    results = {}

    # Test case 1: 2x2 identity matrix
    A1 = torch.eye(2, device='cuda')
    results["test_case_1"] = det(A1).item()

    # Test case 2: 3x3 matrix with random values
    A2 = torch.rand((3, 3), device='cuda')
    results["test_case_2"] = det(A2).item()

    # Test case 3: 4x4 matrix with all zeros
    A3 = torch.zeros((4, 4), device='cuda')
    results["test_case_3"] = det(A3).item()

    # Test case 4: 2x2 matrix with specific values
    A4 = torch.tensor([[4.0, 7.0], [2.0, 6.0]], device='cuda')
    results["test_case_4"] = det(A4).item()

    return results

test_results = test_det()
```

0.6.4. Why The I/O Test Passes

- candidate stdout: ''
- reference stdout: ''
- the stdout strings are equal, so the weak oracle accepts the generated file;
- strict comparison then reports: \$: type differs: NoneType != dict.

0.6.5. Why It Is Not Correct

The gold is determinant with PyTorch’s batched and singular-matrix semantics. Generated code may compute via a restricted LU/ log-det path and miss signs, batches, or singular behavior.

0.6.6. Obvious Counterexample / Follow-up Test

Use batched matrices including a negative determinant and a singular matrix; compare torch.det results including zero/signed behavior.

0.6.7. All Stdout-False-Positive Candidates For This Task

- Bench_T_coder_specific/qwen_full/linalg.py: \$: type differs: NoneType != dict
- Bench_T_coder_specific/qwen_full_rag/linalg.py: \$: type differs: NoneType != dict
- Bench_T_general_purpose/output_DeepSeek-R1/linalg.py: \$: type differs: NoneType != dict
- Bench_T_general_purpose/output_DeepSeek-R1_rag/linalg.py: \$: type differs: NoneType != dict
- Bench_T_general_purpose/output_claude-3-5-sonnet-20240620/linalg.py: \$: type differs: NoneType != dict
- Bench_T_general_purpose/output_claude-3-5-sonnet-20240620_rag/linalg.py: \$: type differs: NoneType != dict
- Bench_T_general_purpose/output_gpt-4o/linalg.py: \$: type differs: NoneType != dict
- Bench_T_general_purpose/output_gpt-4o_rag/linalg.py: \$: type differs: NoneType != dict
- Bench_T_general_purpose/output_o1-2024-12-17/linalg.py: \$: type differs: NoneType != dict
- Bench_T_general_purpose/output_o1-2024-12-17_rag/linalg.py: \$: type differs: NoneType != dict

0.7. T / dropout_sigmoid_linear

field	value
-------	-------

speedup	1.6489
stdout false-positive candidates	14
representative candidate	Bench_T_general_purpose/output_DeepSeek-R1/def_0094.py
strict mismatch	\$. type differs: NoneType != dict
candidate stdout	''
reference stdout	''

0.7.1. GT Implementation Snippet

```
import torch
import torch.nn.functional as F

def dropout_sigmoid_linear(input: torch.Tensor, weight: torch.Tensor, bias=None, p=0.5, training=True,
inplace=False) -> torch.Tensor:
    """
    Applies a linear transformation followed by a sigmoid activation and dropout.

    Args:
        input (torch.Tensor): Input tensor of shape (*, in_features).
        weight (torch.Tensor): Weight tensor of shape (out_features, in_features).
        bias (torch.Tensor, optional): Bias tensor of shape (out_features). Default: None.
        p (float, optional): Probability of an element to be zeroed in dropout. Default: 0.5.
        training (bool, optional): If True, applies dropout during training. Default: True.
        inplace (bool, optional): If True, performs the operation in-place. Default: False.

    Returns:
        torch.Tensor: The resulting tensor after applying the linear transformation, sigmoid activation, and
        dropout.
    """
    output = F.linear(input, weight, bias)
    output = torch.sigmoid(output)
    if training:
        output = F.dropout(output, p=p, training=training, inplace=inplace)
    return output
```

0.7.2. Generated Solution Snippet

```
import torch
import triton
import triton.language as tl

@triton.jit
def _sigmoid_dropout_kernel(
    input_ptr,
    output_ptr,
    n_elements,
    p,
    training,
    seed,
    BLOCK_SIZE: tl.constexpr,
):
    pid = tl.program_id(axis=0)
    block_start = pid * BLOCK_SIZE
    offsets = block_start + tl.arange(0, BLOCK_SIZE)
    mask = offsets < n_elements

    x = tl.load(input_ptr + offsets, mask=mask)
    x = 1.0 / (1.0 + tl.exp(-x))

    if training:
        rnd = tl.rand(seed, offsets)
        keep = rnd > p
        x = tl.where(keep, x / (1 - p), 0.0)

    tl.store(output_ptr + offsets, x, mask=mask)

def dropout_sigmoid_linear(
    input: torch.Tensor,
    weight: torch.Tensor,
    bias=None,
```



```

    p=0.5,
    training=True,
    inplace=False,
) -> torch.Tensor:
    output = torch.nn.functional.linear(input, weight, bias)
    if inplace:
        output_sigmoid_dropout = output
    else:
        output_sigmoid_dropout = torch.empty_like(output)

    n_elements = output.numel()
    grid = lambda meta: (triton.cdiv(n_elements, meta['BLOCK_SIZE']),)

    max_seed = 2147483647
    seed = torch.randint(0, max_seed, (1,), device=output.device).item()

    _sigmoid_dropout_kernel[grid](
        output,
        output_sigmoid_dropout,
        n_elements,
        p,
        training,
        seed,
        BLOCK_SIZE=1024,
    )
    return output_sigmoid_dropout

```

0.7.3. I/O Test Inputs That Are Treated As Passing

The weak oracle sees this as passing because the generated script and the gold script have identical stdout. The recovered gold test tail below shows the inputs and result dictionary that should have been compared structurally.

```

import torch
import torch.nn.functional as F

# def dropout_sigmoid_linear(input: torch.Tensor, weight: torch.Tensor, bias=None, p=0.5, training=True,
# inplace=False) -> torch.Tensor:
#     """
#     Applies a linear transformation followed by a sigmoid activation and dropout.

#     Args:
#         input (torch.Tensor): Input tensor of shape (*, in_features).
#         weight (torch.Tensor): Weight tensor of shape (out_features, in_features).
#         bias (torch.Tensor, optional): Bias tensor of shape (out_features). Default: None.
#         p (float, optional): Probability of an element to be zeroed in dropout. Default: 0.5.
#         training (bool, optional): If True, applies dropout during training. Default: True.
#         inplace (bool, optional): If True, performs the operation in-place. Default: False.

#     Returns:
#         torch.Tensor: The resulting tensor after applying the linear transformation, sigmoid activation, and
#         dropout.
#     """
#     output = F.linear(input, weight, bias)
#     output = torch.sigmoid(output)
#     if training:
#         output = F.dropout(output, p=p, training=training, inplace=inplace)
#     return output

def test_dropout_sigmoid_linear():
    results = {}

    # Test case 1: Basic test with bias, training=True, inplace=False
    input = torch.randn(2, 3, device='cuda')
    weight = torch.randn(4, 3, device='cuda')
    bias = torch.randn(4, device='cuda')
    results["test_case_1"] = dropout_sigmoid_linear(input, weight, bias)

    # Test case 2: No bias, training=True, inplace=False
    input = torch.randn(2, 3, device='cuda')
    weight = torch.randn(4, 3, device='cuda')
    results["test_case_2"] = dropout_sigmoid_linear(input, weight)

    # Test case 3: With bias, training=False, inplace=False

```

```

input = torch.randn(2, 3, device='cuda')
weight = torch.randn(4, 3, device='cuda')
bias = torch.randn(4, device='cuda')
results["test_case_3"] = dropout_sigmoid_linear(input, weight, bias, training=False)

# Test case 4: With bias, training=True, inplace=True
input = torch.randn(2, 3, device='cuda')
weight = torch.randn(4, 3, device='cuda')
bias = torch.randn(4, device='cuda')
results["test_case_4"] = dropout_sigmoid_linear(input, weight, bias, inplace=True)

return results

```

```
test_results = test_dropout_sigmoid_linear()
```

0.7.4. Why The I/O Test Passes

- candidate stdout: ''
- reference stdout: ''
- the stdout strings are equal, so the weak oracle accepts the generated file;
- strict comparison then reports: \$: type differs: NoneType != dict.

0.7.5. Why It Is Not Correct

Dropout is stochastic in training and identity in eval. Generated candidates often hard-code or mishandle dropout masks/scaling, and stdout equality cannot observe returned tensors.

0.7.6. Obvious Counterexample / Follow-up Test

Check training=False and p=0 first, then seeded training=True, p=0.5 with PyTorch's expected dropout scaling.

0.7.7. All Stdout-False-Positive Candidates For This Task

- Bench_T_coder_specific/deepseek/def_0094.py: \$: type differs: NoneType != dict
- Bench_T_coder_specific/deepseek_full/def_0094.py: \$: type differs: NoneType != dict
- Bench_T_coder_specific/deepseek_full_rag/def_0094.py: \$: type differs: NoneType != dict
- Bench_T_coder_specific/deepseek_rag/def_0094.py: \$: type differs: NoneType != dict
- Bench_T_coder_specific/qwen/def_0094.py: \$: type differs: NoneType != dict
- Bench_T_general_purpose/output_DeepSeek-R1/def_0094.py: \$: type differs: NoneType != dict
- Bench_T_general_purpose/output_DeepSeek-R1_rag/def_0094.py: \$: type differs: NoneType != dict
- Bench_T_general_purpose/output_Qwen2.5-72B-Instruct/def_0094.py: \$: type differs: NoneType != dict
- Bench_T_general_purpose/output_claude-3-5-sonnet-20240620/def_0094.py: \$: type differs: NoneType != dict
- Bench_T_general_purpose/output_claude-3-5-sonnet-20240620_rag/def_0094.py: \$: type differs: NoneType != dict
- Bench_T_general_purpose/output_gpt-4o/def_0094.py: \$: type differs: NoneType != dict
- Bench_T_general_purpose/output_gpt-4o_rag/def_0094.py: \$: type differs: NoneType != dict
- Bench_T_general_purpose/output_o1-2024-12-17/def_0094.py: \$: type differs: NoneType != dict
- Bench_T_general_purpose/output_o1-2024-12-17_rag/def_0094.py: \$: type differs: NoneType != dict

0.8. T / elu_linear

field	value
speedup	1.9
stdout false-positive candidates	7
representative candidate	Bench_T_general_purpose/output_o1-2024-12-17_rag/def_0146.py
strict mismatch	\$: type differs: NoneType != dict
candidate stdout	''
reference stdout	''

0.8.1. GT Implementation Snippet

```

import torch
import torch.nn.functional as F

```

```
def elu_linear(input, weight, bias=None, alpha=1.0, inplace=False):
    """
    Applies a linear transformation followed by the Exponential Linear Unit (ELU) activation.

    Parameters:
        input (Tensor): The input tensor for the linear layer.
                        Shape should be (batch_size, in_features).
        weight (Tensor): The weight tensor for the linear transformation.
                        Shape should be (out_features, in_features).
        bias (Tensor, optional): The bias tensor for the linear transformation. Default: None.
                        Shape should be (out_features).
        alpha (float, optional): The  $\alpha$  parameter for the ELU function. Default: 1.0.
        inplace (bool, optional): Whether to apply ELU in-place. Default: False.

    Returns:
        Tensor: The output tensor after applying the linear transformation and ELU activation.
                Shape will be (batch_size, out_features).
    """
    output = F.linear(input, weight, bias)
    return F.elu(output, alpha=alpha, inplace=inplace)
```

0.8.2. Generated Solution Snippet

```
import torch
import triton
import triton.language as tl

# Triton kernel for ELU activation
@triton.jit
def elu_kernel(
    x_ptr,          # *Pointer* to input vector
    alpha_ptr,      # *Pointer* to alpha scalar
    output_ptr,     # *Pointer* to output vector
    n_elements,     # Total number of elements
    BLOCK_SIZE: tl.constexpr
):
    pid = tl.program_id(axis=0)
    block_start = pid * BLOCK_SIZE
    offsets = block_start + tl.arange(0, BLOCK_SIZE)
    mask = offsets < n_elements

    x = tl.load(x_ptr + offsets, mask=mask)
    alpha = tl.load(alpha_ptr)
    out = tl.where(x > 0, x, alpha * (tl.exp(x) - 1))
    tl.store(output_ptr + offsets, out, mask=mask)

def elu(x: torch.Tensor, alpha: torch.Tensor) -> torch.Tensor:
    output = torch.empty_like(x)
    n_elements = x.numel()
    grid = lambda meta: (triton.cdiv(n_elements, meta['BLOCK_SIZE']),)
    elu_kernel[grid](x, alpha, output, n_elements, BLOCK_SIZE=1024)
    return output

def elu_inplace(x: torch.Tensor, alpha: torch.Tensor):
    n_elements = x.numel()
    grid = lambda meta: (triton.cdiv(n_elements, meta['BLOCK_SIZE']),)
    elu_kernel[grid](x, alpha, x, n_elements, BLOCK_SIZE=1024)

def elu_linear(input: torch.Tensor,
               weight: torch.Tensor,
               bias: torch.Tensor = None,
               alpha: float = 1.0,
               inplace: bool = False) -> torch.Tensor:
    # Linear transformation
    out = input.matmul(weight)
    if bias is not None:
        out = out + bias

    # Prepare alpha as a single-element tensor on the same device
```

```

alpha_t = torch.tensor([alpha], dtype=out.dtype, device=out.device)

# ELU activation
if inplace:
    elu_inplace(out, alpha_t)
    return out
else:
    return elu(out, alpha_t)

```

0.8.3. I/O Test Inputs That Are Treated As Passing

The weak oracle sees this as passing because the generated script and the gold script have identical stdout. The recovered gold test tail below shows the inputs and result dictionary that should have been compared structurally.

```

import torch
import torch.nn.functional as F

# def elu_linear(input, weight, bias=None, alpha=1.0, inplace=False):
#     output = F.linear(input, weight, bias)
#     return F.elu(output, alpha=alpha, inplace=inplace)

def test_elu_linear():
    results = {}

    # Test case 1: Basic test with bias, alpha=1.0, inplace=False
    input1 = torch.tensor([[1.0, 2.0], [3.0, 4.0]], device='cuda')
    weight1 = torch.tensor([[0.5, -0.5], [-0.5, 0.5]], device='cuda')
    bias1 = torch.tensor([0.0, 0.0], device='cuda')
    results["test_case_1"] = elu_linear(input1, weight1, bias1)

    # Test case 2: Without bias, alpha=1.0, inplace=False
    input2 = torch.tensor([[1.0, 2.0], [3.0, 4.0]], device='cuda')
    weight2 = torch.tensor([[0.5, -0.5], [-0.5, 0.5]], device='cuda')
    results["test_case_2"] = elu_linear(input2, weight2)

    # Test case 3: With bias, alpha=0.5, inplace=False
    input3 = torch.tensor([[1.0, 2.0], [3.0, 4.0]], device='cuda')
    weight3 = torch.tensor([[0.5, -0.5], [-0.5, 0.5]], device='cuda')
    bias3 = torch.tensor([0.0, 0.0], device='cuda')
    results["test_case_3"] = elu_linear(input3, weight3, bias3, alpha=0.5)

    # Test case 4: With bias, alpha=1.0, inplace=True
    input4 = torch.tensor([[1.0, 2.0], [3.0, 4.0]], device='cuda')
    weight4 = torch.tensor([[0.5, -0.5], [-0.5, 0.5]], device='cuda')
    bias4 = torch.tensor([0.0, 0.0], device='cuda')
    results["test_case_4"] = elu_linear(input4, weight4, bias4, inplace=True)

    return results

test_results = test_elu_linear()

```

0.8.4. Why The I/O Test Passes

- candidate stdout: ''
- reference stdout: ''
- the stdout strings are equal, so the weak oracle accepts the generated file;
- strict comparison then reports: \$: type differs: NoneType != dict.

0.8.5. Why It Is Not Correct

The gold applies a linear operation and ELU with PyTorch's shape, bias, alpha, and optional output semantics. Generated candidates often flatten or specialize the matrix case.

0.8.6. Obvious Counterexample / Follow-up Test

Use bias=None, non-default alpha, and a non-contiguous input; compare to the PyTorch reference exactly.

0.8.7. All Stdout-False-Positive Candidates For This Task

- Bench_T_coder_specific/deepseek_full/def_0146.py: \$: type differs: NoneType != dict
- Bench_T_general_purpose/output_DeepSeek-R1/def_0146.py: \$: type differs: NoneType != dict
- Bench_T_general_purpose/output_Qwen2.5-72B-Instruct/def_0146.py: \$: type differs: NoneType != dict
- Bench_T_general_purpose/output_claude-3-5-sonnet-20240620/def_0146.py: \$: type differs: NoneType != dict

- Bench_T_general_purpose/output_gpt-4o/def_0146.py: \$: type differs: NoneType != dict
- Bench_T_general_purpose/output_o1-2024-12-17/def_0146.py: \$: type differs: NoneType != dict
- Bench_T_general_purpose/output_o1-2024-12-17_rag/def_0146.py: \$: type differs: NoneType != dict

0.9. T / fused_cross_entropy_log_softmax

field	value
speedup	1.4365
stdout false-positive candidates	9
representative candidate	Bench_T_general_purpose/output_o1-2024-12-17/def_0078.py
strict mismatch	\$: type differs: NoneType != dict
candidate stdout	''
reference stdout	''

0.9.1. GT Implementation Snippet

```
import torch
import torch.nn.functional as F

def fused_cross_entropy_log_softmax(input: torch.Tensor, target: torch.Tensor, dim: int=1, weight:
torch.Tensor=None, ignore_index: int=-100, reduction: str='mean', label_smoothing: float=0.0) -> torch.Tensor:
    """
    Computes the cross entropy loss with log softmax applied to the input logits.

    Args:
        input (Tensor): Input tensor of logits, where softmax will be computed along `dim`.
        target (Tensor): Ground truth class indices or probabilities.
        dim (int, optional): Dimension along which to compute log softmax. Default is 1.
        weight (Tensor, optional): Manual rescaling weight for each class.
        ignore_index (int, optional): Specifies a target value that is ignored and does not contribute to the
input gradient. Default: -100.
        reduction (str, optional): Specifies the reduction to apply to the output: 'none' | 'mean' | 'sum'.
Default: 'mean'.
        label_smoothing (float, optional): Specifies the amount of smoothing to be applied, where 0.0 means no
smoothing. Default: 0.0.

    Returns:
        Tensor: The computed loss.
    """
    log_probs = F.log_softmax(input, dim=dim)
    loss = F.cross_entropy(log_probs, target, weight=weight, ignore_index=ignore_index, reduction=reduction,
label_smoothing=label_smoothing)
    return loss
```

0.9.2. Generated Solution Snippet

```
import triton
import triton.language as tl
import torch

@triton.jit
def _fused_cross_entropy_log_softmax_kernel(
    input_ptr, target_ptr, output_ptr,
    N, C,
    stride_n, stride_c,
    ignore_index,
    label_smoothing,
    use_weight,
    weight_ptr,
    BLOCK_SIZE: tl.constexpr
):
    pid_n = tl.program_id(0)
    # Each block processes one row (batch element)
    # If pid_n >= N, just return
    if pid_n >= N:
        return

    # Pointers to the row we are processing
    row_input_ptr = input_ptr + pid_n * stride_n
```

```

row_target = tl.load(target_ptr + pid_n)
# Skip row if ignore_index is set and matched
skip_row = (row_target == ignore_index)

# 1) Compute row max for numerical stability
# We'll fetch in increments of BLOCK_SIZE
idxs = tl.arange(0, BLOCK_SIZE)
max_val = tl.float32(-1e30)
# We'll loop over columns with step BLOCK_SIZE
for start_c in range(0, C, BLOCK_SIZE):
    mask = idxs + start_c < C
    row_val = tl.where(
        mask,
        tl.load(row_input_ptr + (start_c + idxs) * stride_c),
        # Large negative for masked out
        tl.float32(-1e30)
    )
    block_max = tl.maximum(tl.max(row_val, axis=0), max_val)
    max_val = block_max
# Broadcast final max to all threads in block
row_max = tl.max(max_val, axis=0)

# 2) Compute sum of exp(values - row_max) and optional sum of input for label smoothing
sum_exp = tl.float32(0.)
sum_input = tl.float32(0.)
for start_c in range(0, C, BLOCK_SIZE):
    mask = idxs + start_c < C
    row_val = tl.where(
        mask,
        tl.load(row_input_ptr + (start_c + idxs) * stride_c),
        0.0
    )
    shifted = row_val - row_max
    exp_val = tl.exp(shifted)
    sum_exp += tl.sum(tl.where(mask, exp_val, 0.), axis=0)
    sum_input += tl.sum(tl.where(mask, row_val, 0.), axis=0)

log_sum_exp = tl.log(sum_exp)

# 3) Compute the cross entropy for this row
# We'll do partial computations for label smoothing if not skipping
loss_val = tl.float32(0.)
if skip_row:
    # If target is to be ignored, set loss to 0
    loss_val = 0.
else:
    # Confirm valid target in [0, C)
    # If it's out of range, we can set 0 as well
    row_target = tl.where(
        (row_target >= 0) & (row_target < C),
        row_target,
        -1
    )
    # We'll find log_smax for the correct class and also sum log_smax if label_smoothing
    correct_col = row_target.to(tl.int32)
    # We'll do a second pass to gather log_smax_correct and sum_log_smax
    sum_log_smax = tl.float32(0.)
    log_smax_correct = tl.float32(0.)

    running_correct = tl.float32(0.)
    running_sum_log_smax = tl.float32(0.)

    for start_c in range(0, C, BLOCK_SIZE):
        mask = idxs + start_c < C
        row_val = tl.where(
            mask,
            tl.load(row_input_ptr + (start_c + idxs) * stride_c),
            ... <truncated; 99 more lines>

```

0.9.3. I/O Test Inputs That Are Treated As Passing

The weak oracle sees this as passing because the generated script and the gold script have identical stdout. The recovered gold test tail below shows the inputs and result dictionary that should have been compared structurally.

```

import torch
import torch.nn.functional as F

def test_fused_cross_entropy_log_softmax():
    results = {}

    # Test case 1: Basic test with default parameters
    input = torch.tensor([[1.0, 2.0, 3.0], [1.0, 2.0, 3.0]], device='cuda')
    target = torch.tensor([2, 1], device='cuda')
    results["test_case_1"] = fused_cross_entropy_log_softmax(input, target)

    # Test case 2: Test with label smoothing
    input = torch.tensor([[1.0, 2.0, 3.0], [1.0, 2.0, 3.0]], device='cuda')
    target = torch.tensor([2, 1], device='cuda')
    results["test_case_2"] = fused_cross_entropy_log_softmax(input, target, label_smoothing=0.1)

    # Test case 3: Test with weight
    input = torch.tensor([[1.0, 2.0, 3.0], [1.0, 2.0, 3.0]], device='cuda')
    target = torch.tensor([2, 1], device='cuda')
    weight = torch.tensor([1.0, 0.5, 2.0], device='cuda')
    results["test_case_3"] = fused_cross_entropy_log_softmax(input, target, weight=weight)

    # Test case 4: Test with sum reduction
    input = torch.tensor([[1.0, 2.0, 3.0], [1.0, 2.0, 3.0]], device='cuda')
    target = torch.tensor([2, 1], device='cuda')
    results["test_case_4"] = fused_cross_entropy_log_softmax(input, target, reduction='sum')

    return results

test_results = test_fused_cross_entropy_log_softmax()

```

0.9.4. Why The I/O Test Passes

- candidate stdout: ''
- reference stdout: ''
- the stdout strings are equal, so the weak oracle accepts the generated file;
- strict comparison then reports: \$: type differs: NoneType != dict.

0.9.5. Why It Is Not Correct

The gold combines cross entropy and log softmax with PyTorch semantics, including weights, ignore_index, and reduction. Generated implementations frequently divide weighted mean losses by the wrong denominator.

0.9.6. Obvious Counterexample / Follow-up Test

Use class weights, ignore_index, and reduction='mean'; the mean must divide by the sum of selected target weights, not merely by the count.

0.9.7. All Stdout-False-Positive Candidates For This Task

- Bench_T_coder_specific/deepseek/def_0078.py: \$: type differs: NoneType != dict
- Bench_T_coder_specific/qwen_full/def_0078.py: \$: type differs: NoneType != dict
- Bench_T_general_purpose/output_DeepSeek-R1/def_0078.py: \$: type differs: NoneType != dict
- Bench_T_general_purpose/output_DeepSeek-R1_rag/def_0078.py: \$: type differs: NoneType != dict
- Bench_T_general_purpose/output_Qwen2.5-72B-Instruct_rag/def_0078.py: \$: type differs: NoneType != dict
- Bench_T_general_purpose/output_claude-3-5-sonnet-20240620_rag/def_0078.py: \$: type differs: NoneType != dict
- Bench_T_general_purpose/output_gpt-4o/def_0078.py: \$: type differs: NoneType != dict
- Bench_T_general_purpose/output_o1-2024-12-17/def_0078.py: \$: type differs: NoneType != dict
- Bench_T_general_purpose/output_o1-2024-12-17_rag/def_0078.py: \$: type differs: NoneType != dict

0.10. T / fused_qr_solve

field	value
speedup	1.7538
stdout false-positive candidates	12
representative candidate	Bench_T_coder_specific/deepseek_full_rag/def_0080.py

strict mismatch	\$: type differs: NoneType != dict
candidate stdout	''
reference stdout	''

0.10.1. GT Implementation Snippet

```
import torch

def fused_qr_solve(A: torch.Tensor, b: torch.Tensor) -> torch.Tensor:
    """
    Solve the linear system Ax = b using QR decomposition.

    Args:
        A (torch.Tensor): The matrix A of shape (m, n) where m >= n.
        b (torch.Tensor): The right-hand side tensor b of shape (m, k).

    Returns:
        torch.Tensor: The solution tensor x of shape (n, k).
    """
    (Q, R) = torch.linalg.qr(A)
    Qt_b = torch.matmul(Q.T, b)
    x = torch.linalg.solve(R, Qt_b)
    return x
```

0.10.2. Generated Solution Snippet

```
import torch
import triton
import triton.language as tl

@triton.jit
def fused_qr_solve_kernel(
    A, # shape: (m, n)
    b, # shape: (m, k)
    m,
    n,
    k,
    q, # shape: (m, m)
    tau, # shape: (m,)
    x, # shape: (n, k)
    BLOCK_M: tl.constexpr,
    BLOCK_N: tl.constexpr,
    BLOCK_K: tl.constexpr,
):
    # Triton kernel implementation
    pass

def fused_qr_solve(A: torch.Tensor, b: torch.Tensor):
    # Function to call the Triton kernel
    m, n = A.shape
    assert m >= n
    k = b.shape[1]
    assert A.shape == (m, n)
    assert b.shape == (m, k)
    q, _ = torch.linalg.qr(A)
    # Use torch.org_qr to get the same result as torch.linalg.qr
    tau = torch.empty((m,), dtype=torch.float32, device=A.device)
    torch.org_qr(A, tau)
    x = torch.empty((n, k), dtype=torch.float32, device=A.device)
    # Call the Triton kernel
    fused_qr_solve_kernel[(m, k)](
        A, b, m, n, k, q, tau, x, BLOCK_M=128, BLOCK_N=128, BLOCK_K=32
    )
    return x
```

0.10.3. I/O Test Inputs That Are Treated As Passing

The weak oracle sees this as passing because the generated script and the gold script have identical stdout. The recovered gold test tail below shows the inputs and result dictionary that should have been compared structurally.

```
import torch

def test_fused_qr_solve():
```



```

results = {}

# Test case 1: Square matrix A and vector b
A1 = torch.tensor([[1.0, 2.0], [3.0, 4.0]], device='cuda')
b1 = torch.tensor([5.0], [6.0]], device='cuda')
results["test_case_1"] = fused_qr_solve(A1, b1)

# Test case 2: Rectangular matrix A (m > n) and vector b
A2 = torch.tensor([[1.0, 2.0], [3.0, 4.0], [5.0, 6.0]], device='cuda')
b2 = torch.tensor([7.0], [8.0], [9.0]], device='cuda')
results["test_case_2"] = fused_qr_solve(A2, b2)

# Test case 3: Square matrix A and matrix b with multiple columns
A3 = torch.tensor([[1.0, 2.0], [3.0, 4.0]], device='cuda')
b3 = torch.tensor([5.0, 6.0], [7.0, 8.0]], device='cuda')
results["test_case_3"] = fused_qr_solve(A3, b3)

# Test case 4: Rectangular matrix A (m > n) and matrix b with multiple columns
A4 = torch.tensor([[1.0, 2.0], [3.0, 4.0], [5.0, 6.0]], device='cuda')
b4 = torch.tensor([7.0, 8.0], [9.0, 10.0], [11.0, 12.0]], device='cuda')
results["test_case_4"] = fused_qr_solve(A4, b4)

return results

test_results = test_fused_qr_solve()

```

0.10.4. Why The I/O Test Passes

- candidate stdout: ''
- reference stdout: ''
- the stdout strings are equal, so the weak oracle accepts the generated file;
- strict comparison then reports: \$: type differs: NoneType != dict.

0.10.5. Why It Is Not Correct

The gold solves a linear system via QR and returns a numerical solution. Several generated files are incomplete, use placeholders, or do not expose a result dictionary under the strict checker.

0.10.6. Obvious Counterexample / Follow-up Test

Use a full-rank rectangular A and b, then check the residual norm($A @ x - b$) against the gold solution.

0.10.7. All Stdout-False-Positive Candidates For This Task

- Bench_T_coder_specific/deepseek/def_0080.py: \$: type differs: NoneType != dict
- Bench_T_coder_specific/deepseek_full/def_0080.py: \$: type differs: NoneType != dict
- Bench_T_coder_specific/deepseek_full_rag/def_0080.py: \$: type differs: NoneType != dict
- Bench_T_coder_specific/qwen_rag/def_0080.py: \$: type differs: NoneType != dict
- Bench_T_general_purpose/output_DeepSeek-R1/def_0080.py: \$: type differs: NoneType != dict
- Bench_T_general_purpose/output_DeepSeek-R1_rag/def_0080.py: \$: type differs: NoneType != dict
- Bench_T_general_purpose/output_Qwen2.5-72B-Instruct/def_0080.py: \$: type differs: NoneType != dict
- Bench_T_general_purpose/output_Qwen2.5-72B-Instruct_rag/def_0080.py: \$: type differs: NoneType != dict
- Bench_T_general_purpose/output_claude-3-5-sonnet-20240620/def_0080.py: \$: type differs: NoneType != dict
- Bench_T_general_purpose/output_claude-3-5-sonnet-20240620_rag/def_0080.py: \$: type differs: NoneType != dict
- Bench_T_general_purpose/output_gpt-4o/def_0080.py: \$: type differs: NoneType != dict
- Bench_T_general_purpose/output_gpt-4o_rag/def_0080.py: \$: type differs: NoneType != dict

0.11. T / fused_transformer_block

field	value
speedup	2.2619
stdout false-positive candidates	11
representative candidate	Bench_T_general_purpose/output_o1-2024-12-17_rag/fused_transformer_block.py
strict mismatch	\$: type differs: NoneType != dict

candidate stdout	''
reference stdout	''

0.11.1. GT Implementation Snippet

```
import torch
import torch.nn.functional as F

def fused_transformer_block(input, weight1, weight2, residual, dropout_p=0.1, eps=1e-05, *, out=None):
    """
    Performs a sequence of operations commonly used in transformer models.

    Arguments:
    - input (Tensor): Input tensor of shape (*, N, D_in), where * denotes any number of batch dimensions.
    - weight1 (Tensor): Weight matrix of shape (D_in, D_k).
    - weight2 (Tensor): Weight matrix of shape (D_k, D_out).
    - residual (Tensor): Residual tensor to be added before layer normalization, must be broadcastable to the
    shape of Z_4.
    - dropout_p (float, optional): Probability of an element to be zeroed in the dropout layer. Default: 0.1.
    - eps (float, optional): A value added to the denominator for numerical stability in layer normalization.
    Default: 1e-5.
    - out (Tensor, optional): Output tensor. Ignored if None. Default: None.

    Returns:
    - Tensor: The output tensor after performing the sequence of operations.
    """
    z1 = input @ weight1
    z2 = F.softmax(z1, dim=-1)
    z3 = F.dropout(z2, p=dropout_p, training=True)
    z4 = z3 @ weight2
    y = F.layer_norm(z4 + residual, normalized_shape=(z4.size(-1)), eps=eps)
    if out is not None:
        out.copy_(y)
    return out
    return y
```

0.11.2. Generated Solution Snippet

```
import torch
import torch.nn.functional as F
import triton
import triton.language as tl

@triton.jit
def _layernorm_fwd_kernel(
    x_ptr, # [M, N] input
    y_ptr, # [M, N] output
    stride_x, # stride for x
    stride_y, # stride for y
    N, # number of columns (normalized dimension)
    eps, # epsilon for numerical stability
    BLOCK_SIZE: tl.constexpr,
):
    """
    A Triton kernel that applies layer normalization over the last dimension.
    Computation is done for one row per program_id(0).
    """
    row_id = tl.program_id(0)
    x_row_ptr = x_ptr + row_id * stride_x
    y_row_ptr = y_ptr + row_id * stride_y

    # 1) Compute mean and sum of squares in a first pass
    # We'll accumulate partial sums of x and x^2 in registers
    sum_x = tl.zeros([BLOCK_SIZE], dtype=tl.float32)
    sum_x2 = tl.zeros([BLOCK_SIZE], dtype=tl.float32)
    for offset in range(0, N, BLOCK_SIZE):
        cols = offset + tl.arange(0, BLOCK_SIZE)
        mask = cols < N
        x = tl.load(x_row_ptr + cols, mask=mask, other=0.0).to(tl.float32)
        sum_x += x
        sum_x2 += x * x

    # Reduce partial sums across the BLOCK_SIZE vector
```

```

mean_val = tl.sum(sum_x) / N
mean_x2 = tl.sum(sum_x2) / N
var_val = mean_x2 - mean_val * mean_val
var_val = tl.where(var_val > 0.0, var_val, 0.0)
rstd_val = 1.0 / tl.sqrt(var_val + eps)

# 2) Apply normalization in a second pass
for offset in range(0, N, BLOCK_SIZE):
    cols = offset + tl.arange(0, BLOCK_SIZE)
    mask = cols < N
    x = tl.load(x_row_ptr + cols, mask=mask, other=0.0).to(tl.float32)
    x_hat = (x - mean_val) * rstd_val
    tl.store(y_row_ptr + cols, x_hat, mask=mask)

def _layernorm_triton(input_tensor: torch.Tensor, eps: float):
    """
    Applies layer normalization over the last dimension of input_tensor using the Triton kernel.
    Gamma=1 and Beta=0 are assumed (i.e., no scale or bias).
    """
    # Flatten all dimensions except the last
    # Suppose input_tensor has shape (*, N), we flatten into [M, N]
    x_resaped = input_tensor.reshape(-1, input_tensor.shape[-1])
    M, N = x_resaped.shape
    # Prepare output
    y_resaped = torch.empty_like(x_resaped)

    # Determine BLOCK_SIZE
    BLOCK_SIZE = triton.next_power_of_2(N)
    # Ensure BLOCK_SIZE is not excessively large:
    BLOCK_SIZE = max(32, min(BLOCK_SIZE, 1024))

    # Enqueue the Triton kernel
    grid = (M,)
    _layernorm_fwd_kernel[grid](
        x_resaped,          # x_ptr
        y_resaped,          # y_ptr
        x_resaped.stride(0), # stride_x
        y_resaped.stride(0), # stride_y
        N,
        eps,
        BLOCK_SIZE=BLOCK_SIZE,
    )

    # Reshape back to original
    return y_resaped.reshape_as(input_tensor)

def fused_transformer_block(input, weight1, weight2, residual, dropout_p=0.1, eps=1e-5, *, out=None):
    """
    fused_transformer_block(input, weight1, weight2, residual, dropout_p=0.1, eps=1e-5, *, out=None) -> Tensor

    Performs a sequence of operations commonly used in a transformer block:

        1) Z1 = input @ weight1
    ... <truncated>; 38 more lines>

```

0.11.3. I/O Test Inputs That Are Treated As Passing

The weak oracle sees this as passing because the generated script and the gold script have identical stdout. The recovered gold test tail below shows the inputs and result dictionary that should have been compared structurally.

```

import torch
import torch.nn.functional as F

def test_fused_transformer_block():
    results = {}

    # Test case 1: Basic functionality test
    input1 = torch.randn(2, 3, 4, device='cuda')
    weight1_1 = torch.randn(4, 5, device='cuda')
    weight2_1 = torch.randn(5, 4, device='cuda')
    residual1 = torch.randn(2, 3, 4, device='cuda')

```

```

results["test_case_1"] = fused_transformer_block(input1, weight1_1, weight2_1, residual1)

# Test case 2: Different input size
input2 = torch.randn(1, 5, 6, device='cuda')
weight1_2 = torch.randn(6, 7, device='cuda')
weight2_2 = torch.randn(7, 6, device='cuda')
residual2 = torch.randn(1, 5, 6, device='cuda')
results["test_case_2"] = fused_transformer_block(input2, weight1_2, weight2_2, residual2)

# Test case 3: Test with dropout probability set to 0
input3 = torch.randn(3, 2, 4, device='cuda')
weight1_3 = torch.randn(4, 5, device='cuda')
weight2_3 = torch.randn(5, 4, device='cuda')
residual3 = torch.randn(3, 2, 4, device='cuda')
results["test_case_3"] = fused_transformer_block(input3, weight1_3, weight2_3, residual3, dropout_p=0.0)

# Test case 4: Test with a different epsilon value
input4 = torch.randn(4, 3, 5, device='cuda')
weight1_4 = torch.randn(5, 6, device='cuda')
weight2_4 = torch.randn(6, 5, device='cuda')
residual4 = torch.randn(4, 3, 5, device='cuda')
results["test_case_4"] = fused_transformer_block(input4, weight1_4, weight2_4, residual4, eps=1e-3)

return results

test_results = test_fused_transformer_block()

```

0.11.4. Why The I/O Test Passes

- candidate stdout: ''
- reference stdout: ''
- the stdout strings are equal, so the weak oracle accepts the generated file;
- strict comparison then reports: \$: type differs: NoneType != dict.

0.11.5. Why It Is Not Correct

The gold block includes linear/normalization/dropout-style behavior with specific argument semantics. Generated versions often simplify stochastic dropout or partial block behavior, which stdout cannot observe.

0.11.6. Obvious Counterexample / Follow-up Test

Set training=False and p=0 to remove randomness, compare deterministic output; then test seeded dropout separately.

0.11.7. All Stdout-False-Positive Candidates For This Task

- Bench_T_coder_specific/deepseek_full/fused_transformer_block.py: \$: type differs: NoneType != dict
- Bench_T_coder_specific/qwen_full/fused_transformer_block.py: \$: type differs: NoneType != dict
- Bench_T_coder_specific/qwen_full_rag/fused_transformer_block.py: \$: type differs: NoneType != dict
- Bench_T_general_purpose/output_DeepSeek-R1/fused_transformer_block.py: \$: type differs: NoneType != dict
- Bench_T_general_purpose/output_DeepSeek-R1_rag/fused_transformer_block.py: \$: type differs: NoneType != dict
- Bench_T_general_purpose/output_Qwen2.5-72B-Instruct_rag/fused_transformer_block.py: \$: type differs: NoneType != dict
- Bench_T_general_purpose/output_claude-3-5-sonnet-20240620_rag/fused_transformer_block.py: \$: type differs: NoneType != dict
- Bench_T_general_purpose/output_gpt-4o/fused_transformer_block.py: \$: type differs: NoneType != dict
- Bench_T_general_purpose/output_gpt-4o_rag/fused_transformer_block.py: \$: type differs: NoneType != dict
- Bench_T_general_purpose/output_o1-2024-12-17/fused_transformer_block.py: \$: type differs: NoneType != dict
- Bench_T_general_purpose/output_o1-2024-12-17_rag/fused_transformer_block.py: \$: type differs: NoneType != dict

0.12. T / invert_matrix_lu

field	value
speedup	1.6266
stdout false-positive candidates	7
representative candidate	Bench_T_general_purpose/output_DeepSeek-R1_rag/invert_matrix_lu.py

strict mismatch	\$: type differs: NoneType != dict
candidate stdout	''
reference stdout	''

0.12.1. GT Implementation Snippet

```
import torch

def invert_matrix_lu(A, *, pivot=True, out=None):
    """
    Computes the inverse of a square matrix using LU decomposition.

    Parameters:
        A (Tensor): A square invertible matrix.
        pivot (bool, optional): Whether to use partial pivoting (default: True).
        out (Tensor, optional): An output tensor to store the result (default: None).

    Returns:
        Tensor: The inverse of matrix A.
    """
    (P, L, U) = torch.linalg.lu(A, pivot=pivot)
    n = A.size(-1)
    if pivot:
        P_eye = torch.eye(n, device=A.device, dtype=A.dtype).expand_as(A)
        P_mat = P @ P_eye
    else:
        P_mat = torch.eye(n, device=A.device, dtype=A.dtype)
    Y = torch.linalg.solve(L, P_mat)
    A_inv = torch.linalg.solve(U, Y)
    if out is not None:
        out.copy_(A_inv)
        return out
    return A_inv
```

0.12.2. Generated Solution Snippet

```
import torch
import triton
import triton.language as tl

@triton.jit
def lower_triangular_solve_kernel(
    L_ptr, B_ptr, Y_ptr, n,
    stride_L_batch, stride_L_row, stride_L_col,
    stride_B_batch, stride_B_row, stride_B_col,
    stride_Y_batch, stride_Y_row, stride_Y_col,
    BLOCK_SIZE: tl.constexpr
):
    pid_batch = tl.program_id(0)
    pid_col = tl.program_id(1)

    for i in range(n):
        sum_val = 0.0
        for j in range(i):
            l_ij = tl.load(L_ptr + pid_batch * stride_L_batch + i * stride_L_row + j * stride_L_col)
            y_jk = tl.load(Y_ptr + pid_batch * stride_Y_batch + j * stride_Y_row + pid_col * stride_Y_col)
            sum_val += l_ij * y_jk
        b_ik = tl.load(B_ptr + pid_batch * stride_B_batch + i * stride_B_row + pid_col * stride_B_col)
        y_ik = b_ik - sum_val
        tl.store(Y_ptr + pid_batch * stride_Y_batch + i * stride_Y_row + pid_col * stride_Y_col, y_ik)

@triton.jit
def upper_triangular_solve_kernel(
    U_ptr, Y_ptr, X_ptr, n,
    stride_U_batch, stride_U_row, stride_U_col,
    stride_Y_batch, stride_Y_row, stride_Y_col,
    stride_X_batch, stride_X_row, stride_X_col,
    BLOCK_SIZE: tl.constexpr
):
    pid_batch = tl.program_id(0)
    pid_col = tl.program_id(1)
```

```

for i in range(n-1, -1, -1):
    sum_val = 0.0
    for j in range(i+1, n):
        u_ij = tl.load(U_ptr + pid_batch * stride_U_batch + i * stride_U_row + j * stride_U_col)
        x_jk = tl.load(X_ptr + pid_batch * stride_X_batch + j * stride_X_row + pid_col * stride_X_col)
        sum_val += u_ij * x_jk
    y_ik = tl.load(Y_ptr + pid_batch * stride_Y_batch + i * stride_Y_row + pid_col * stride_Y_col)
    u_ii = tl.load(U_ptr + pid_batch * stride_U_batch + i * stride_U_row + i * stride_U_col)
    x_ik = (y_ik - sum_val) / u_ii
    tl.store(X_ptr + pid_batch * stride_X_batch + i * stride_X_row + pid_col * stride_X_col, x_ik)

def invert_matrix_lu(A: torch.Tensor, *, pivot=True, out=None) -> torch.Tensor:
    assert A.shape[-1] == A.shape[-2], "A must be a square matrix or batches of square matrices"
    if A.dtype not in [torch.float32, torch.float64, torch.complex64, torch.complex128]:
        raise ValueError(f"Unsupported dtype {A.dtype}")

    batch_dims = A.shape[:-2]
    n = A.size(-1)
    batch_size = int(torch.prod(torch.tensor(batch_dims))) if batch_dims else 1

    if pivot:
        LU, pivots = torch.lu(A, get_infos=True)
        P, L, U = torch.lu_unpack(LU, pivots)
    else:
        raise NotImplementedError("LU without pivoting is not implemented in this solution")

    I = torch.eye(n, dtype=A.dtype, device=A.device).expand_as(A)
    B = P @ I

    Y = torch.empty_like(B)
    grid = (batch_size, n)
    L = L.contiguous()
    B = B.contiguous()
    Y = Y.contiguous()

    lower_triangular_solve_kernel[grid](
        L, B, Y, n,
        L.stride(-3) if L.ndim > 2 else 0, L.stride(-2), L.stride(-1),
        B.stride(-3) if B.ndim > 2 else 0, B.stride(-2), B.stride(-1),
        Y.stride(-3) if Y.ndim > 2 else 0, Y.stride(-2), Y.stride(-1),
        BLOCK_SIZE=128
    )

    X = torch.empty_like(Y)
    U = U.contiguous()
    X = X.contiguous()

    upper_triangular_solve_kernel[grid](
        U, Y, X, n,
        U.stride(-3) if U.ndim > 2 else 0, U.stride(-2), U.stride(-1),
        Y.stride(-3) if Y.ndim > 2 else 0, Y.stride(-2), Y.stride(-1),
        X.stride(-3) if X.ndim > 2 else 0, X.stride(-2), X.stride(-1),
        BLOCK_SIZE=128
    )
... <truncated; 4 more lines>

```

0.12.3. I/O Test Inputs That Are Treated As Passing

The weak oracle sees this as passing because the generated script and the gold script have identical stdout. The recovered gold test tail below shows the inputs and result dictionary that should have been compared structurally.

```

import torch

def test_invert_matrix_lu():
    results = {}

    # Test case 1: Basic test with pivot=True
    A1 = torch.tensor([[4.0, 3.0], [6.0, 3.0]], device='cuda')
    results["test_case_1"] = invert_matrix_lu(A1)

    # Test case 2: Basic test with pivot=False
    A2 = torch.tensor([[4.0, 3.0], [6.0, 3.0]], device='cuda')
    results["test_case_2"] = invert_matrix_lu(A2, pivot=False)

```

```

# Test case 3: Larger matrix with pivot=True
A3 = torch.tensor([[7.0, 2.0, 1.0], [0.0, 3.0, -1.0], [-3.0, 4.0, 2.0]], device='cuda')
results["test_case_3"] = invert_matrix_lu(A3)

# Test case 4: Larger matrix with pivot=False
A4 = torch.tensor([[7.0, 2.0, 1.0], [0.0, 3.0, -1.0], [-3.0, 4.0, 2.0]], device='cuda')
results["test_case_4"] = invert_matrix_lu(A4, pivot=False)

return results

test_results = test_invert_matrix_lu()

```

0.12.4. Why The I/O Test Passes

- candidate stdout: ''
- reference stdout: ''
- the stdout strings are equal, so the weak oracle accepts the generated file;
- strict comparison then reports: \$: type differs: NoneType != dict.

0.12.5. Why It Is Not Correct

The gold computes matrix inverse through LU semantics. Generated files can pass empty stdout while not proving correct inverses, pivot handling, or batched behavior.

0.12.6. Obvious Counterexample / Follow-up Test

Use a batched well-conditioned matrix and verify both $A @ \text{inv}(A)$ and $\text{inv}(A) @ A$ are identity within tolerance.

0.12.7. All Stdout-False-Positive Candidates For This Task

- Bench_T_coder_specific/qwen_rag/invert_matrix_lu.py: \$: type differs: NoneType != dict
- Bench_T_general_purpose/output_DeepSeek-R1/invert_matrix_lu.py: \$: type differs: NoneType != dict
- Bench_T_general_purpose/output_DeepSeek-R1_rag/invert_matrix_lu.py: \$: type differs: NoneType != dict
- Bench_T_general_purpose/output_Qwen2.5-72B-Instruct/invert_matrix_lu.py: \$: type differs: NoneType != dict
- Bench_T_general_purpose/output_Qwen2.5-72B-Instruct_rag/invert_matrix_lu.py: \$: type differs: NoneType != dict
- Bench_T_general_purpose/output_claude-3-5-sonnet-20240620/invert_matrix_lu.py: \$: type differs: NoneType != dict
- Bench_T_general_purpose/output_o1-2024-12-17/invert_matrix_lu.py: \$: type differs: NoneType != dict

0.13. T / lu.py

field	value
speedup	1.1829
stdout false-positive candidates	10
representative candidate	Bench_T_general_purpose/output_DeepSeek-R1/lu.py
strict mismatch	\$: type differs: NoneType != dict
candidate stdout	''
reference stdout	''

0.13.1. GT Implementation Snippet

```

import torch

def lu(A, pivot=True, out=None):
    """
    Computes the LU decomposition of a matrix (or batch of matrices) using torch.linalg.lu.

    Args:
        A (Tensor): Input tensor of shape `(*, m, n)`, where `*` represents zero or more batch dimensions.
        pivot (bool, optional): If True, performs LU decomposition with partial pivoting. Default is True.
        out (tuple, optional): Tuple of three tensors to store the output. Defaults to None.

    Returns:
        tuple: (P, L, U) where:
            - P is the permutation matrix (only if pivot=True),
            - L is the lower triangular matrix (with ones on the diagonal),
    
```

- U is the upper triangular matrix.

Example:

```
>>> A = torch.randn(3, 2)
>>> P, L, U = compute_lu_decomposition(A)
>>> P
tensor([[0., 1., 0.],
        [0., 0., 1.],
        [1., 0., 0.]])
>>> L
tensor([[1.0000, 0.0000],
        [0.5007, 1.0000],
        [0.0633, 0.9755]])
>>> U
tensor([[0.3771, 0.0489],
        [0.0000, 0.9644]])
>>> torch.dist(A, P @ L @ U)
tensor(5.9605e-08)

>>> A = torch.randn(2, 5, 7, device="cuda")
>>> P, L, U = compute_lu_decomposition(A, pivot=False)
>>> P
tensor([], device='cuda:0')
>>> torch.dist(A, L @ U)
tensor(1.0376e-06, device='cuda:0')
```

"""

```
(P, L, U) = torch.linalg.lu(A, pivot=pivot, out=out)
return (P, L, U)
```

0.13.2. Generated Solution Snippet

```
import torch
import triton
import triton.language as tl

@triton.jit
def lu_no_pivot_kernel(
    a_ptr, l_ptr, u_ptr,
    m, n,
    stride_a_batch, stride_a_row, stride_a_col,
    stride_l_batch, stride_l_row, stride_l_col,
    stride_u_batch, stride_u_row, stride_u_col,
    BLOCK_SIZE: tl.constexpr,
):
    pid_batch = tl.program_id(0)

    a_offset = pid_batch * stride_a_batch
    l_offset = pid_batch * stride_l_batch
    u_offset = pid_batch * stride_u_batch

    # Copy A to U and initialize L as identity
    for i in range(m):
        for j in range(n):
            u_p = u_offset + i * stride_u_row + j * stride_u_col
            a_p = a_offset + i * stride_a_row + j * stride_a_col
            val = tl.load(a_ptr + a_p)
            tl.store(u_ptr + u_p, val)
        for j in range(m):
            for j in range(m):
                l_p = l_offset + i * stride_l_row + j * stride_l_col
                val = 1.0 if i == j else 0.0
                tl.store(l_ptr + l_p, val)

    for k in range(min(m, n)):
        u_kk_p = u_offset + k * stride_u_row + k * stride_u_col
        u_kk = tl.load(u_ptr + u_kk_p)
        for i in range(k + 1, m):
            l_ik_p = l_offset + i * stride_l_row + k * stride_l_col
            u_ik_p = u_offset + i * stride_u_row + k * stride_u_col
            u_ik = tl.load(u_ptr + u_ik_p)
            l_ik = u_ik / u_kk
            tl.store(l_ptr + l_ik_p, l_ik)
            tl.store(u_ptr + u_ik_p, 0.0)
```



```

        for j in range(k + 1, n):
            u_ij_p = u_offset + i * stride_u_row + j * stride_u_col
            u_kj_p = u_offset + k * stride_u_row + j * stride_u_col
            u_ij = tl.load(u_ptr + u_ij_p)
            u_kj = tl.load(u_ptr + u_kj_p)
            u_ij_updated = u_ij - l_ik * u_kj
            tl.store(u_ptr + u_ij_p, u_ij_updated)

def lu(A, *, pivot=True, out=None):
    assert A.dim() >= 2, "A must have at least 2 dimensions"
    m, n = A.shape[-2], A.shape[-1]
    batch_dims = A.shape[:-2]
    device = A.device

    if pivot:
        lu_tensor, pivots = torch.lu(A, get_infos=False)
        P, L, U = torch.lu_unpack(lu_tensor, pivots)
        return (P, L, U)
    else:
        if not A.is_cuda:
            raise NotImplementedError("LU without pivoting is only supported on GPU")

        L = torch.empty(*batch_dims, m, m, dtype=A.dtype, device=device)
        U = torch.empty_like(A)
        num_batches = 1
        for dim in batch_dims:
            num_batches *= dim

        BLOCK_SIZE = 16 # Tune this based on hardware

        lu_no_pivot_kernel[(num_batches,)](
            A, L, U,
            m, n,
            A.stride(-3) if A.dim() >= 3 else 0,
            A.stride(-2), A.stride(-1),
            L.stride(-3) if L.dim() >= 3 else 0,
            L.stride(-2), L.stride(-1),
            U.stride(-3) if U.dim() >= 3 else 0,
            U.stride(-2), U.stride(-1),
            BLOCK_SIZE=BLOCK_SIZE,
        )
        P = torch.tensor([], dtype=A.dtype, device=device)
        return (P, L, U)

```

0.13.3. I/O Test Inputs That Are Treated As Passing

The weak oracle sees this as passing because the generated script and the gold script have identical stdout. The recovered gold test tail below shows the inputs and result dictionary that should have been compared structurally.

```

import torch

def test_lu():
    results = {}

    # Test case 1: 2x2 matrix with pivoting
    A1 = torch.randn(2, 2, device="cuda")
    P1, L1, U1 = lu(A1)
    results["test_case_1"] = (P1.cpu(), L1.cpu(), U1.cpu())

    # Test case 2: 3x3 matrix with pivoting
    A2 = torch.randn(3, 3, device="cuda")
    P2, L2, U2 = lu(A2)
    results["test_case_2"] = (P2.cpu(), L2.cpu(), U2.cpu())

    # Test case 3: 2x3 matrix without pivoting
    A3 = torch.randn(2, 3, device="cuda")
    P3, L3, U3 = lu(A3, pivot=False)
    results["test_case_3"] = (P3.cpu(), L3.cpu(), U3.cpu())

    # Test case 4: Batch of 2x2 matrices with pivoting
    A4 = torch.randn(4, 2, 2, device="cuda")
    P4, L4, U4 = lu(A4)
    results["test_case_4"] = (P4.cpu(), L4.cpu(), U4.cpu())

```

```

    return results

test_results = test_lu()

```

0.13.4. Why The I/O Test Passes

- candidate stdout: ''
- reference stdout: ''
- the stdout strings are equal, so the weak oracle accepts the generated file;
- strict comparison then reports: \$: type differs: NoneType != dict.

0.13.5. Why It Is Not Correct

The gold exposes PyTorch LU behavior including pivoting modes and returned pivots/info. Generated wrappers often only cover the simplest pivoted case.

0.13.6. Obvious Counterexample / Follow-up Test

Exercise pivot=False when supported and compare the full tuple, including pivots/info, not just the reconstructed matrix.

0.13.7. All Stdout-False-Positive Candidates For This Task

- Bench_T_coder_specific/qwen_full/lu.py: \$: type differs: NoneType != dict
- Bench_T_coder_specific/qwen_rag/lu.py: \$: type differs: NoneType != dict
- Bench_T_general_purpose/output_DeepSeek-R1/lu.py: \$: type differs: NoneType != dict
- Bench_T_general_purpose/output_DeepSeek-R1_rag/lu.py: \$: type differs: NoneType != dict
- Bench_T_general_purpose/output_Qwen2.5-72B-Instruct/lu.py: \$: type differs: NoneType != dict
- Bench_T_general_purpose/output_Qwen2.5-72B-Instruct_rag/lu.py: \$: type differs: NoneType != dict
- Bench_T_general_purpose/output_claude-3-5-sonnet-20240620/lu.py: \$: type differs: NoneType != dict
- Bench_T_general_purpose/output_claude-3-5-sonnet-20240620_rag/lu.py: \$: type differs: NoneType != dict
- Bench_T_general_purpose/output_gpt-4o/lu.py: \$: type differs: NoneType != dict
- Bench_T_general_purpose/output_o1-2024-12-17/lu.py: \$: type differs: NoneType != dict

0.14. T / sigmoid_adaptive_avg_pool2d

field	value
speedup	1.4444
stdout false-positive candidates	9
representative candidate	Bench_T_coder_specific/deepseek_full/def_0081.py
strict mismatch	\$: type differs: NoneType != dict
candidate stdout	''
reference stdout	''

0.14.1. GT Implementation Snippet

```

import torch
import torch.nn.functional as F
from torch import Tensor
from typing import Union, Tuple

def sigmoid_adaptive_avg_pool2d(input: Tensor, output_size: Union[int, Tuple[int, int]]) -> Tensor:
    """
    Applies a 2D adaptive average pooling over an input tensor, followed by the sigmoid activation function
    applied element-wise.

    Args:
        input (Tensor): The input tensor with shape (batch_size, channels, height, width).
        output_size (Union[int, Tuple[int, int]]): The target output size of the pooled tensor.

    Returns:
        Tensor: The result tensor after applying adaptive average pooling and sigmoid activation.
    """
    pooled_output = F.adaptive_avg_pool2d(input, output_size)

```

```

output = torch.sigmoid(pooled_output)
return output

```

0.14.2. Generated Solution Snippet

```

import torch
import triton
import triton.language as tl
from torch import Tensor
from typing import Union, Tuple

@triton.jit
def sigmoid(x):
    # Sigmoid function:  $S(x) = 1 / (1 + \exp(-x))$ 
    return 1 / (1 + tl.exp(-x))

@triton.jit
def adaptive_avg_pool2d(input, output_size):
    # AdaptiveAvgPool2D function
    # Implementation omitted for brevity
    pass

@triton.jit
def sigmoid_adaptive_avg_pool2d(input, output_size):
    # sigmoid_adaptive_avg_pool2d function
    out = adaptive_avg_pool2d(input, output_size)
    return sigmoid(out)

def test_sigmoid_adaptive_avg_pool2d(func_inputs):
    input, output_size = func_inputs
    triton_out = sigmoid_adaptive_avg_pool2d(input, output_size)
    torch_out = torch.sigmoid(input.adaptive_avg_pool2d(output_size))
    assert torch_out.shape == triton_out.shape
    assert torch_out.device == triton_out.device
    assert torch_out.stride() == triton_out.stride()
    assert torch_out.is_contiguous() == triton_out.is_contiguous()
    assert torch_out.is_contiguous() == triton_out.is_contiguous()
    assert torch_out.requires_grad == triton_out.requires_grad
    assert torch_out.grad_fn is None if triton_out.grad_fn is None else triton_out.grad_fn.name() ==
"sigmoid_backward"
    assert torch_out.dtype == triton_out.dtype
    assert torch_out.layout == triton_out.layout
    assert torch_out.ndim == triton_out.ndim
    assert torch_out.real == triton_out.real
    assert torch_out.imag == triton_out.imag
    assert torch_out.numel() == triton_out.numel()
    assert torch_out.tolist() == triton_out.tolist()
    assert torch_out.item() == triton_out.item()
    assert torch_out.shape == triton_out.shape
    assert torch_out.stride() == triton_out.stride()
    assert torch_out.storage_offset() == triton_out.storage_offset()
    assert torch_out.contiguous().stride() == triton_out.contiguous().stride()
    assert torch_out.is_contiguous() == triton_out.is_contiguous()
    assert torch_out.is_contiguous(1) == triton_out.is_contiguous(1)
    assert torch_out.is_contiguous(2) == triton_out.is_contiguous(2)
    assert torch_out.is_contiguous(as_strided=(True, True))
    assert torch_out.ndim == 1
    assert torch_out.size(0) == triton_out.size(0)
    assert torch_out.stride(0) == triton_out.stride(0)
    assert torch_out.storage_offset() == triton_out.storage_offset()
    assert torch_out.is_floating_point() == triton_out.is_floating_point()
    assert torch_out.is_signed() == triton_out.is_signed()
    assert torch_out.is_complex() == triton_out.is_complex()
    assert torch_out.is_quasiconjugate() == triton_out.is_quasiconjugate()
    assert torch_out.is_sparse == triton_out.is_sparse
    assert torch_out.is_sparse_csr == triton_out.is_sparse_csr
    assert torch_out.is_sparse_coo == triton_out.is_sparse_coo
    assert torch_out.is_sparse_bsr == triton_out.is_sparse_bsr
    assert torch_out.is_mkl_tensor() == triton_out.is_mkl_tensor()
    assert torch_out.mkl_lowp_dtype() == triton_out.mkl_lowp_dtype()
    assert torch_out.is_non_blocking() == triton_out.is_non_blocking()
    assert torch_out.is_pinned() == triton_out.is_pinned()
    assert torch_out.is_distributed() == triton_out.is_distributed()

```

```

assert torch_out.get_device() == triton_out.get_device()
assert torch_out.get_device_index() == triton_out.get_device_index()
assert torch_out.get_dtype() == triton_out.get_dtype()
assert torch_out.dim() == triton_out.dim()
assert torch_out.numel() == triton_out.numel()
assert torch_out.size() == triton_out.size()
assert torch_out.requires_grad == triton_out.requires_grad
assert torch_out.grad is None if triton_out.grad is None else torch_out.grad.stride() ==
triton_out.grad.stride()
assert torch_out.grad_fn is None if triton_out.grad_fn is None else torch_out.grad_fn.name ==
triton_out.grad_fn.name
assert torch_out.layout == triton_out.layout
assert torch_out.device == triton_out.device
assert torch_out.dtype == triton_out.dtype
assert torch_out.is_floating_point() == triton_out.is_floating_point()
assert torch_out.is_signed() == triton_out.is_signed()
assert torch_out.is_complex() == triton_out.is_complex()
assert torch_out.is_quasiconjugate() == triton_out.is_quasiconjugate()
assert torch_out.is_sparse == triton_out.is_sparse
assert torch_out.is_sparse_csr == triton_out.is_sparse_csr
assert torch_out.is_sparse_coo == triton_out.is_sparse_coo
assert torch_out.is_sparse_bsr == triton_out.is_sparse_bsr
assert torch_out.is_mkl_tensor() == triton_out.is_mkl_tensor()
assert torch_out.mkl_lowp_dtype() == triton_out.mkl_lowp_dtype()
assert torch_out.is_non_blocking() == triton_out.is_non_blocking()
... <truncated; 6 more lines>

```

0.14.3. I/O Test Inputs That Are Treated As Passing

The weak oracle sees this as passing because the generated script and the gold script have identical stdout. The recovered gold test tail below shows the inputs and result dictionary that should have been compared structurally.

```

def test_sigmoid_adaptive_avg_pool2d():
    # Initialize a dictionary to store the results of each test case
    results = {}

    # Test case 1: Basic test with a 4D tensor and output size as an integer
    input_tensor1 = torch.randn(1, 3, 8, 8, device='cuda') # Batch size 1, 3 channels, 8x8 size
    output_size1 = 4
    result1 = sigmoid_adaptive_avg_pool2d(input_tensor1, output_size1)
    results["test_case_1"] = result1

    # Test case 2: Test with a 4D tensor and output size as a tuple
    input_tensor2 = torch.randn(2, 3, 10, 10, device='cuda') # Batch size 2, 3 channels, 10x10 size
    output_size2 = (5, 5)
    result2 = sigmoid_adaptive_avg_pool2d(input_tensor2, output_size2)
    results["test_case_2"] = result2

    # Test case 3: Test with a larger batch size
    input_tensor3 = torch.randn(4, 3, 16, 16, device='cuda') # Batch size 4, 3 channels, 16x16 size
    output_size3 = (8, 8)
    result3 = sigmoid_adaptive_avg_pool2d(input_tensor3, output_size3)
    results["test_case_3"] = result3

    # Test case 4: Test with a single channel
    input_tensor4 = torch.randn(1, 1, 12, 12, device='cuda') # Batch size 1, 1 channel, 12x12 size
    output_size4 = (6, 6)
    result4 = sigmoid_adaptive_avg_pool2d(input_tensor4, output_size4)
    results["test_case_4"] = result4

    return results

test_results = test_sigmoid_adaptive_avg_pool2d()

```

0.14.4. Why The I/O Test Passes

- candidate stdout: ''
- reference stdout: ''
- the stdout strings are equal, so the weak oracle accepts the generated file;
- strict comparison then reports: \$: type differs: NoneType != dict.

0.14.5. Why It Is Not Correct

The gold computes `torch.sigmoid(F.adaptive_avg_pool2d(input, output_size))`. A representative generated Triton version passes `input.data_ptr()` integers into a kernel launch and uses `Python range(h_start, h_end)` where `h_start/h_end` are Triton values, so the kernel is not a valid implementation.

0.14.6. Obvious Counterexample / Follow-up Test

Use `x = torch.randn(1, 3, 8, 8, device='cuda'), output_size=4`. The gold returns shape `(1,3,4,4)`; the representative generated code fails to launch/compile correctly.

0.14.7. All Stdout-False-Positive Candidates For This Task

- `Bench_T_coder_specific/deepseek_full/def_0081.py`: \$: type differs: NoneType != dict
- `Bench_T_coder_specific/qwen_full/def_0081.py`: \$: type differs: NoneType != dict
- `Bench_T_general_purpose/output_DeepSeek-R1/def_0081.py`: \$: type differs: NoneType != dict
- `Bench_T_general_purpose/output_DeepSeek-R1_rag/def_0081.py`: \$: type differs: NoneType != dict
- `Bench_T_general_purpose/output_claude-3-5-sonnet-20240620/def_0081.py`: \$: type differs: NoneType != dict
- `Bench_T_general_purpose/output_claude-3-5-sonnet-20240620_rag/def_0081.py`: \$: type differs: NoneType != dict
- `Bench_T_general_purpose/output_gpt-4o/def_0081.py`: \$: type differs: NoneType != dict
- `Bench_T_general_purpose/output_o1-2024-12-17/def_0081.py`: \$: type differs: NoneType != dict
- `Bench_T_general_purpose/output_o1-2024-12-17_rag/def_0081.py`: \$: type differs: NoneType != dict

0.15. T / sum_std

field	value
speedup	1.0316
stdout false-positive candidates	6
representative candidate	<code>Bench_T_coder_specific/deepseek_full_rag/def_0052.py</code>
strict mismatch	<code>\$: type differs: NoneType != dict</code>
candidate stdout	<code>''</code>
reference stdout	<code>''</code>

0.15.1. GT Implementation Snippet

```
import torch
```

```
def sum_std(input, dim=None, keepdim=False, dtype=None, correction=1, out=None):
    """
    Computes the sum of elements in the input tensor along the specified dimension(s),
    followed by calculating the standard deviation of the summed values.

    Args:
        input (Tensor): The input tensor.
        dim (int or tuple of ints, optional): The dimension(s) to reduce. If None, all dimensions are reduced.
        keepdim (bool, optional): Whether the output tensor has dim retained or not. Default is False.
        dtype (torch.dtype, optional): The desired data type of the returned tensor. Default: None.
        correction (int, optional): Difference between the sample size and sample degrees of freedom. Default is 1.

    1. out (Tensor, optional): The output tensor.

    Returns:
        Tensor: A tensor containing the standard deviation of the summed values along the specified dimension(s).
    """
    summed = input.sum(dim=dim, keepdim=keepdim, dtype=dtype)
    n = summed.numel()
    mean = summed.mean()
    var = ((summed - mean) ** 2).sum()
    if n > correction:
        std = (var / (n - correction)).sqrt()
    else:
        std = torch.tensor(0.0, dtype=summed.dtype)
    return std
```

0.15.2. Generated Solution Snippet

```
import torch
import triton
import triton.language as tl
from torch import Tensor

@triton.jit
def sum_std_kernel(input, out, correction, N, BLOCK_N: tl.constexpr):
    # Triton kernel to compute sum and standard deviation
    pid = tl.program_id(0)
    offset = pid * BLOCK_N + tl.arange(0, BLOCK_N)
    mask = offset < N
    x = tl.load(input + offset, mask=mask).to(tl.float32)
    x_mean = tl.sum(x, axis=0) / N
    x_centered = x - x_mean
    x_var = tl.sum(x_centered * x_centered, axis=0) / tl.maximum(0, N - correction)
    x_std = tl.sqrt(x_var)
    tl.store(out, x_std)

def sum_std(input: Tensor, dim=None, keepdim=False, dtype=None, correction=1, out=None) -> Tensor:
    # Wrapper function to compute sum and standard deviation
    if dtype is None:
        dtype = input.dtype
    if out is None:
        out = torch.empty(input.shape[:input.ndim - len(dim)], dtype=dtype, device=input.device)
    dim = sorted([int(d) % input.ndim for d in dim]) if dim is not None else list(range(input.ndim))
    N = list(input.shape)[input.ndim - len(dim)] if dim is not None else input.numel()
    input = input.reshape(-1, N)
    input = torch.t(input)
    BLOCK_N = triton.next_power_of_2(input.shape[0])
    grid = (1,)
    sum_std_kernel[grid](input, out, correction, input.shape[0], BLOCK_N=BLOCK_N)
    return out
```

0.15.3. I/O Test Inputs That Are Treated As Passing

The weak oracle sees this as passing because the generated script and the gold script have identical stdout. The recovered gold test tail below shows the inputs and result dictionary that should have been compared structurally.

```
import torch

def test_sum_std():
    results = {}

    # Test case 1: Basic test with a 1D tensor
    input1 = torch.tensor([1.0, 2.0, 3.0, 4.0], device='cuda')
    results["test_case_1"] = sum_std(input1)

    # Test case 2: Test with a 2D tensor along dim=0
    input2 = torch.tensor([[1.0, 2.0], [3.0, 4.0]], device='cuda')
    results["test_case_2"] = sum_std(input2, dim=0)

    # Test case 3: Test with a 2D tensor along dim=1
    input3 = torch.tensor([[1.0, 2.0], [3.0, 4.0]], device='cuda')
    results["test_case_3"] = sum_std(input3, dim=1)

    # Test case 4: Test with keepdim=True
    input4 = torch.tensor([[1.0, 2.0], [3.0, 4.0]], device='cuda')
    results["test_case_4"] = sum_std(input4, dim=0, keepdim=True)

    return results

test_results = test_sum_std()
```

0.15.4. Why The I/O Test Passes

- candidate stdout: ''
- reference stdout: ''
- the stdout strings are equal, so the weak oracle accepts the generated file;
- strict comparison then reports: \$: type differs: NoneType != dict.

0.15.5. Why It Is Not Correct

The gold computes `summed = input.sum(...)` and then `std` over that result. A representative candidate reshapes the scalar `std` to `[1] * input.ndim` when `keepdim=True`; for larger reductions it also stores only `pid == 0`, ignoring later blocks.

0.15.6. Obvious Counterexample / Follow-up Test

Use `x = tensor([[1.,2.],[3.,4.]], device='cuda')` with `dim=0, keepdim=True`. The gold `std` is scalar-shaped, while the representative candidate returns shape `(1, 1)`.

0.15.7. All Stdout-False-Positive Candidates For This Task

- `Bench_T_coder_specific/deepseek_full/def_0052.py`: \$: type differs: NoneType != dict
- `Bench_T_coder_specific/deepseek_full_rag/def_0052.py`: \$: type differs: NoneType != dict
- `Bench_T_general_purpose/output_DeepSeek-R1/def_0052.py`: \$: type differs: NoneType != dict
- `Bench_T_general_purpose/output_DeepSeek-R1_rag/def_0052.py`: \$: type differs: NoneType != dict
- `Bench_T_general_purpose/output_Qwen2.5-72B-Instruct/def_0052.py`: \$: type differs: NoneType != dict
- `Bench_T_general_purpose/output_gpt-4o/def_0052.py`: \$: type differs: NoneType != dict

0.16. T / tensordot_rsqrt

field	value
speedup	2.2318
stdout false-positive candidates	16
representative candidate	<code>Bench_T_coder_specific/deepseek/def_0092.py</code>
strict mismatch	<code>\$: type differs: NoneType != dict</code>
candidate stdout	<code>''</code>
reference stdout	<code>''</code>

0.16.1. GT Implementation Snippet

```
import torch

def tensordot_rsqrt(a: torch.Tensor, b: torch.Tensor, dims) -> torch.Tensor:
    """
    Perform a tensor contraction between two tensors a and b over specified dimensions and apply
    reciprocal square root to the resulting tensor.

    Args:
        a (torch.Tensor): Left tensor to contract.
        b (torch.Tensor): Right tensor to contract.
        dims (int, Tuple[List[int], List[int]], or List[List[int]]): Dimensions for contraction,
            as per `torch.tensordot`.

    Returns:
        torch.Tensor: The reciprocal square root of the tensordot product of tensors a and b.
    """
    result = torch.tensordot(a, b, dims=dims)
    return torch.rsqrt(result)
```

0.16.2. Generated Solution Snippet

```
import triton
import torch

@triton.jit
def tensordot_rsqrt(a, b, dims):
    # Perform tensor contraction
    tensor_contract = torch.tensordot(a, b, dims)

    # Apply reciprocal square root
    output = torch.rsqrt(tensor_contract)

    return output
```

0.16.3. I/O Test Inputs That Are Treated As Passing

The weak oracle sees this as passing because the generated script and the gold script have identical stdout. The recovered gold test tail below shows the inputs and result dictionary that should have been compared structurally.

```

import torch

def test_tensordot_rsqr():
    results = {}

    # Test case 1: Simple contraction with scalar result
    a = torch.tensor([1.0, 2.0, 3.0], device='cuda')
    b = torch.tensor([4.0, 5.0, 6.0], device='cuda')
    dims = 1
    results["test_case_1"] = tensordot_rsqr(a, b, dims)

    # Test case 2: Contraction with matrices
    a = torch.tensor([[1.0, 2.0], [3.0, 4.0]], device='cuda')
    b = torch.tensor([[5.0, 6.0], [7.0, 8.0]], device='cuda')
    dims = ([1], [0])
    results["test_case_2"] = tensordot_rsqr(a, b, dims)

    # Test case 3: Higher-dimensional tensors
    a = torch.tensor([[[[1.0, 2.0], [3.0, 4.0]], [[5.0, 6.0], [7.0, 8.0]]], device='cuda')
    b = torch.tensor([[[[9.0, 10.0], [11.0, 12.0]], [[13.0, 14.0], [15.0, 16.0]]], device='cuda')
    dims = ([2], [1])
    results["test_case_3"] = tensordot_rsqr(a, b, dims)

    # Test case 4: Different dimensions for contraction
    a = torch.tensor([[[1.0, 2.0, 3.0], [4.0, 5.0, 6.0]], device='cuda')
    b = torch.tensor([[7.0, 8.0], [9.0, 10.0], [11.0, 12.0]], device='cuda')
    dims = ([1], [0])
    results["test_case_4"] = tensordot_rsqr(a, b, dims)

    return results

test_results = test_tensordot_rsqr()

```

0.16.4. Why The I/O Test Passes

- candidate stdout: ''
- reference stdout: ''
- the stdout strings are equal, so the weak oracle accepts the generated file;
- strict comparison then reports: \$: type differs: NoneType != dict.

0.16.5. Why It Is Not Correct

The gold computes `torch.rsqr(torch.tensordot(a, b, dims=dims))`. A representative candidate defines a `@triton.jit` function as if it were a normal Python function, uses tensor objects directly inside JIT, and passes `dims` as a Triton reduction axis.

0.16.6. Obvious Counterexample / Follow-up Test

Use `a=[1,2,3]`, `b=[4,5,6]`, `dims=1`; the gold is `rsqr(32)`, while the representative JIT-style candidate is not legally callable as this Python wrapper.

0.16.7. All Stdout-False-Positive Candidates For This Task

- Bench_T_coder_specific/deepseek/def_0092.py: \$: type differs: NoneType != dict
- Bench_T_coder_specific/deepseek_full/def_0092.py: \$: type differs: NoneType != dict
- Bench_T_coder_specific/deepseek_full_rag/def_0092.py: \$: type differs: NoneType != dict
- Bench_T_coder_specific/deepseek_rag/def_0092.py: \$: type differs: NoneType != dict
- Bench_T_coder_specific/qwen_full/def_0092.py: \$: type differs: NoneType != dict
- Bench_T_coder_specific/qwen_full_rag/def_0092.py: \$: type differs: NoneType != dict
- Bench_T_coder_specific/qwen_rag/def_0092.py: \$: type differs: NoneType != dict
- Bench_T_general_purpose/output_DeepSeek-R1/def_0092.py: \$: type differs: NoneType != dict
- Bench_T_general_purpose/output_DeepSeek-R1_rag/def_0092.py: \$: type differs: NoneType != dict
- Bench_T_general_purpose/output_Qwen2.5-72B-Instruct/def_0092.py: \$: type differs: NoneType != dict
- Bench_T_general_purpose/output_Qwen2.5-72B-Instruct_rag/def_0092.py: \$: type differs: NoneType != dict
- Bench_T_general_purpose/output_claude-3-5-sonnet-20240620/def_0092.py: \$: type differs: NoneType != dict
- Bench_T_general_purpose/output_claude-3-5-sonnet-20240620_rag/def_0092.py: \$: type differs: NoneType != dict

- Bench_T_general_purpose/output_gpt-4o/def_0092.py: \$: type differs: NoneType != dict
- Bench_T_general_purpose/output_o1-2024-12-17/def_0092.py: \$: type differs: NoneType != dict
- Bench_T_general_purpose/output_o1-2024-12-17_rag/def_0092.py: \$: type differs: NoneType != dict